

STRESZCZENIE

ABSTRACT

SPIS TREŚCI

Wykaz ważniejszych oznaczeń i skrótów	6
1. Wstęp i cel pracy	8
2. Podział prac	9
3. Wprowadzenie do dziedziny (Karina Wołoszyn)	10
3.1. Rynek gier przeglądarkowych	10
3.2. Rynek gier geolokalizacyjnych	12
3.3. Rodzaje gier konkurencyjnych	13
4. Wykorzystane technologie	15
4.1. Frontendowe (Filip Jezierski)	15
4.1.1. Progressive Web App	15
4.1.2. React	15
4.1.3. Axios	16
4.1.4. Leaflet	16
4.2. Backendowe (Michał Turski)	16
4.2.1. C# ASP .NET Core	16
4.2.2. Wykorzystane pakiety NuGet	16
4.3. Baza danych (Michał Turski)	17
5. Narzędzia	18
5.1. System kontroli wersji (Michał Pawiłojć)	18
5.2. Konteneryzacja - Docker (Michał Pawiłojć)	18
5.2.1. Konteneryzacja	19
5.2.2. Docker	19
5.2.3. Docker Compose	19
5.2.4. Korzyści wynikające z konteneryzacji	19
5.2.5. Ograniczenia	20
5.3. Zarządzanie projektem - YouTrack (Karina Wołoszyn)	20
5.4. Zintegrowane środowisko programistyczne - VSC, VS, Rider (Michał Pawiłojć)	20
5.4.1. Backend: Visual Studio i Rider	21
5.4.2. Frontend: Visual Studio Code	21
5.4.3. Wspólne praktyki i integracja IDE z projektem	21
5.5. Projektowanie interfejsu użytkownika - Figma (Karina Wołoszyn)	22
6. Analiza wymagań	23
6.1. Koncepcja gry (Karina Wołoszyn)	23
6.2. Przebieg gry (Karina Wołoszyn)	23
6.3. Tryby rozgrywki (Karina Wołoszyn)	24
6.4. Wymagania funkcjonalne (Karina Wołoszyn)	24
6.5. Wymagania нефункционалне (Filip Jezierski)	26
6.6. Wybór platformy (Filip Jezierski)	26
6.6.1. Natywna aplikacja mobilna	27
6.6.2. Aplikacja webowa PWA	27

6.7. Ograniczenia projektu (Filip Jezierski)	27
6.7.1. Środowisko aplikacji	27
6.7.2. Zdjęcia lokalizacji	27
6.8. Kryteria akceptacji (Filip Jezierski)	28
7. Projekt systemu (Michał Turski i Filip Jezierski)	29
7.1. Architektura logiki biznesowej (Michał Turski)	29
7.1.1. Wzorzec CQRS	29
7.1.2. Zastosowanie DDD	29
7.2. Baza danych (Michał Turski)	30
7.2.1. Diagram ERD	30
7.2.2. Opis głównych tabel i związków	30
7.3. Analiza interakcji użytkownika z systemem (Michał Turski)	33
7.3.1. Przypadki użycia aplikacji	33
7.3.2. Diagram sekwencji rozgrywki	34
7.4. Estetyka i design (Filip Jezierski)	35
7.4.1. Ogólny projekt interfejsu	35
7.4.2. Kolorystyka	35
7.4.3. Typografia	36
7.4.4. Ikony	36
7.4.5. Responsywność	36
8. Implementacja	38
8.1. Implementacja wyglądu i funkcjonalności frontendu	38
8.1.1. Architektura frontendu (Karina Wołoszyn)	38
8.1.2. Struktura UI i komponentów (Karina Wołoszyn)	38
8.1.3. Warstwa wizualna (Karina Wołoszyn)	38
8.1.4. Obsługa interakcji użytkownika (Karina Wołoszyn)	38
8.1.5. Integracja z geolokalizacją (Filip Jezierski)	38
8.1.6. Komunikacja z backendem (Filip Jezierski)	41
8.1.7. Mechanizmy PWA (Filip Jezierski)	42
8.1.8. Implementacja rozgrywki (Filip Jezierski)	43
8.2. Implementacja backendu i logiki aplikacji	46
8.2.1. Implementacja rozgrywki	46
8.2.2. Autoryzacja i uwierzytelnianie użytkowników	47
8.2.3. Walidacja danych	49
8.2.4. Obsługa błędów w aplikacji	50
8.3. Baza danych i zarządzanie danymi	50
8.3.1. Repozytoria — komunikacja z bazą danych	50
8.3.2. Warstwa persystencji — Entity Framework Core	51
8.3.3. Adminer	51
8.3.4. Problemy napotkane przy implementacji backendu	51
8.4. Devops i wdrożenie aplikacji	51
8.4.1. Środowisko deweloperskie	51
8.4.2. Środowisko produkcyjne	52
8.4.3. Pliki docker-compose i ich role	52
8.4.4. Decyzja o użyciu reverse proxy Nginx	52

8.4.5. Certyfikaty i ich zarządzanie	53
8.4.6. Budowa i publikacja obrazów Docker	53
8.4.7. Konfiguracja aplikacji	53
8.4.8. Przepływ uruchomienia aplikacji	54
8.4.9. Podsumowanie sekcji DevOps	54
8.5. Integracja komponentów systemu	54
9. Testowanie	55
10. Podsumowanie	56
11. Możliwość dalszego rozwoju	57
Wykaz literatury	58
Wykaz rysunków	58
Wykaz tabel	59

WYKAZ WAŻNIEJSZYCH OZNACZEŃ I SKRÓTÓW

- **API** (ang. *Application Programming Interface*) – zestaw reguł i protokołów określających sposób komunikacji oraz integracji pomiędzy różnymi elementami aplikacji.
- **AR** (ang. *Augmented Reality*) – technologia rozszerzonej rzeczywistości, która łączy elementy wirtualne z rzeczywistym światem użytkownika.
- **CI/CD** (ang. *Continuous Integration/Continuous Delivery lub Deployment*) – zestaw praktyk tworzenia oprogramowania
- **CQRS** (ang. *Command Query Responsibility Segregation*) – wzorzec architektoniczny polegający na rozdzieleniu operacji modyfikujących stan systemu od operacji odczytujących dane.
- **DDD** (ang. *Domain-Driven Design*) – podejście do projektowania oprogramowania, które skupia się na odwzorowywaniu rzeczywistego świata w modelu domeny.
- **DOM** (ang. *Document Object Model*) – struktura reprezentująca dokumenty HTML i XML jako hierarchię obiektów, umożliwiającą manipulację ich zawartością i strukturą za pomocą języków programowania.
- **DTO** (ang. *Data Transfer Object*) – obiekt służący do przenoszenia danych między warstwami aplikacji
- **ERD** (ang. *Entity-Relationship Diagram*) – diagram związków encji, służący do modelowania struktury bazy danych
- **GPS** (ang. *Global Positioning System*) – system nawigacji satelitarnej umożliwiający określenie położenia geograficznego na powierzchni Ziemi
- **HTML** (ang. *HyperText Markup Language*) – najnowsza wersja języka znaczników używanego do tworzenia stron internetowych
- **HTTPS** (ang. *HyperText Transfer Protocol Secure*) – bezpieczna wersja protokołu HTTP, która wykorzystuje szyfrowanie SSL/TLS do ochrony przesyłanych danych
- **ITS** (ang. *Issue Tracker System*) – sposób zarządzania systemem, który odpowiada na zapytania wysyłane dowolną drogą.
- **JSON** (ang. *JavaScript Object Notation*) – format wymiany danych oparty na składni języka JavaScript
- **JWT** (ang. *JSON Web Token*) – standardowy format bezpiecznego przesyłania informacji jako obiekt JSON
- **LINQ** (ang. *Language Integrated Query*) – zestawu technologii opartych na integracji możliwości tworzenia zapytań bezpośrednio z językiem C#
- **ORM** (ang. *Object-Relational Mapping*) – technika mapowania danych między systemem obiekowym a relacyjną bazą danych
- **PWA** (ang. *Progressive Web Application*) – aplikacja webowa wykorzystująca nowoczesne technologie webowe w celu zapewnienia doświadczenia podobnego do aplikacji natywnych
- **REST** (ang. *Representational State Transfer*) – styl architektoniczny wykorzystywany przy projektowaniu usług sieciowych

- **SQL** (ang. *Structured Query Language*) – strukturalny język zapytań
- **UI** (ang. *User Interface*) – interfejs użytkownika
- **UX** (ang. *User Experience*) – doświadczenie użytkownika
- **VS** (ang. *Visual Studio*) – zintegrowane środowisko programistyczne (IDE) stworzone przez firmę Microsoft
- **VCS** (ang. *Version Control System*) – system kontroli wersji

1. WSTĘP I CEL PRACY

Celem projektu jest stworzenie gry geograficzno-geolokalizacyjnej polegającej na odgadywaniu miejsc związanych z Politechniką Gdańską na podstawie ich zdjęć. Dodatkowo dostępny będzie tryb terenowy, w którym użytkownik musi fizycznie udać się w odgadywane miejsce. Aby zwiększyć stopień interakcji między użytkownikami, zaplanowaliśmy również udostępnienie publicznego rankingu, zachęcającego do poprawiania swoich wyników, a także dodanie trybu wieloosobowego, w którym gracze rywalizowaliby między sobą, ścigając się o to, kto pierwszy odgadnie zadaną lokalizację.

Aplikacja taka pozwalałaby na interaktywne poznawanie terenów kampusu dla szerokiego grona zainteresowanych, od osób niezwiązanych z uczelnią, do absolwentów oraz pracowników PG. Szczególnie przydatnym zastosowaniem byłoby zapoznanie przyszłych i nowych studentów z okolicami uczelni, ułatwiając im orientację w nowym otoczeniu.

Gry przeglądarkowe cieszyły się popularnością od momentu ich powstania. Ich głównym atutem są niskie wymagania sprzętowe czy brak potrzeby instalacji, które od początku poszerzyły grono potencjalnych odbiorców. Dodatkowo, użytkownicy za pośrednictwem dostępu do internetu mogą wchodzić ze sobą w interakcje i rywalizować dzięki udostępnianym rankingom.

Popularność gier komputerowych znacząco wzrosła podczas pandemii COVID-19, gdzie przez potrzebę pozostania w izolacji możliwości spędzania wolnego czasu były ograniczone. Wydłużony czas pozostania w domach przyczynił się też do wzrostu zainteresowania grami geograficznymi i geolokalizacyjnymi. Jednym z czołowych produktów tego zjawiska został GeoGuessr, który mimo wypuszczenia na rynek 6 lat wcześniej, został spopularyzowany dopiero w 2020 roku. Sukces ten można przypisać idei połączenia rozrywki i odkrywania świata z wykorzystaniem Google Street View, dzięki której użytkownicy mogą się wykazać wiedzą na temat architektury, ukształtowania terenu oraz panującego w danym kraju języka.

Podobnym trendem wykazały się aplikacje mobilne, które ze względu na szybki rozwój urządzeń przenośnych, pozwalają graczowi połączyć się z dowolnego miejsca, w dowolnej chwili. Zlikwidowały one potrzebę posiadania drogiego i nieporęcznego sprzętu, stawiając na wygodę użytkownika. Atuty te wykorzystwała gra Pokemon Go, która z pomocą dobrze znanej marki podbiła rynek, osiągając pułap 500 milionów pobrań w pierwszym roku po debiucie. Gra ta korzysta z funkcji GPS do znalezienia wirtualnych aren rozsianych po prawdziwych lokalizacjach, przyczyniając się do zwiększenia immersji rozrywki.

Po dokonaniu powyższej analizy rynku, postanowiliśmy w ramach projektu inżynierskiego stworzyć aplikację łączącą w sobie dotychczas wymienione aspekty.

2. PODZIAŁ PRAC

3. WPROWADZENIE DO DZIEDZINY (KARINA WOŁOSZYN)

Wzrost popularności gier w ostatnich latach szczególnie uwidocznił się podczas pandemii COVID-19, kiedy miliony ludzi zmieniły swój tryb życia, spędzając większość czasu w domach. Aby zaspokoić nudę w tak ograniczonych warunkach, wielu zwróciło się ku grom - największej spośród form rozrywki. Sytuacja ta przyczyniła się zarówno do powiększenia grupy konsumentów, jak i do wykształcenia nowych nawyków - wzmożonej aktywności w grach, utrzymującej się nawet po powrocie do normalnego stylu życia, bez ograniczeń wynikających z restrykcji. Jednym z gatunków, które zyskały na popularności, były gry przeglądarkowe oraz gry geolokalizacyjne.

3.1. Rynek gier przeglądarkowych

Gry przeglądarkowe to rodzaj gier online, które nie wymagają użycia oddzielnych konsol - do ich uruchomienia wystarczy przeglądarka internetowa, dostępna zarówno na komputerach stacjonarnych, jak i na urządzeniach mobilnych. Charakteryzują się łatwym dostępem, niewielkimi wymaganiami sprzętowymi, brakiem potrzeby instalacji dodatkowego oprogramowania oraz niską ceną lub całkowitą darmowością. Rozgrywka w takich grach jest zazwyczaj podzielona na krótkie sesje, których celem jest przyciągnięcie i utrzymanie uwagi gracza. Popularność gier przeglądarkowych wynika również z możliwości gry wieloosobowej oraz wprowadzenia elementów rywalizacji, takich jak rankingi.

Gry przeglądarkowe istnieją od początku internetu, czyli od lat 90. XX wieku. Szczególną uwagę należy jednak poświęcić latom dwutysięcznym, kiedy firma Adobe Systems przejęła Macromedię i rozwijała produkt Adobe Flash. Flash zrewolucjonizował rynek gier, umożliwiając niezależnym twórcom i małym studiom produkcję niewielkich gier działających na większości komputerów i przeglądarek.

Do typowych gatunków gier przeglądarkowych należą gry MMO (ang. *massively multiplayer online*), w których gracze z całego świata łączą się, aby wspólnie eksplorować i oddawać się immersji w świecie stworzonym przez twórców.



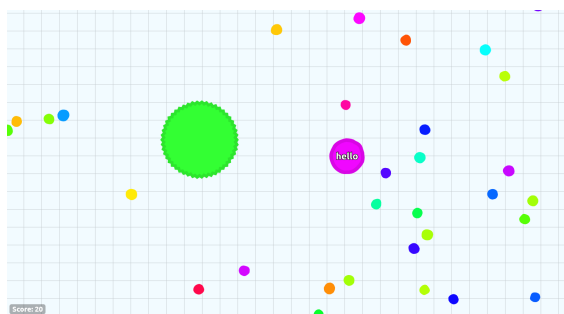
Rysunek 3.1: Gra Travian wydana w 2004 roku.

Gry typu .io to produkcje o prostej mechanice, w których gracze rywalizują online na ograniczonej arenie. Charakteryzują się minimalistyczną grafiką, prostymi zasadami i krótkimi sesjami rozrywki. Popularność tego gatunku zapoczątkowała gra Agar.io, stworzona w 2015 roku przez brazylijskiego



Rysunek 3.2: Gra Club Penguin wydana w 2005 roku.

dewelopera. Gracz steruje w niej jedną (lub wieloma) jednokolorowymi komórkami, których celem jest zdobycie jak największej masy. Aby to osiągnąć, należy unikać większych komórek i zjadać mniejsze.



Rysunek 3.3: Gra Agar.io wydana w 2015 roku.

Na popularności gry Agar.io zyskały również inne produkcje, oparte na podobnej mechanice. W Slither.io gracz steruje wężem, który zwiększa swoją masę poprzez zjadanie komórek. Oprócz tego może pokonać większych rywali, blokując im drogę i eliminując ich, a następnie przejmując ich komórki. Wąż ma również możliwość tymczasowego przyspieszenia - ruch ten pozwala szybciej manewrować po arenie i atakować innych graczy, jednak każdorazowe użycie przyspieszenia powoduje utratę części masy węża, co wymaga strategicznego zarządzania zasobami, aby nie osłabić własnej pozycji.



Rysunek 3.4: Gra Slither.io wydana w 2016 roku.

Wraz z rozwojem przeglądarek gry, które na nich działały, zyskiwały na szybkości przetwarzania animacji i innych elementów graficznych, a także na wydajności i złożoności rozgrywki. Z biegiem lat zmieniały się również sposoby zarządzania pamięcią i obsługi technologii - obok gier działających w

Adobe Flash pojawiły się produkcje oparte na HTML5 (ang. *HyperText Markup Language*), JavaScript i WebGL, a następnie rozwinęto koncepcję PWA (*Progressive Web App*), czyli progresywnych aplikacji umożliwiających działanie zarówno jako natywne aplikacje mobilne, jak i desktopowe.

3.2. Rynek gier geolokalizacyjnych

Gry geolokalizacyjne wykorzystują dane lokalizacyjne gracza oraz informacje satelitarne, łącząc świat rzeczywisty z cyfrowym. Mechanika tych gier opiera się na precyzyjnym wykorzystaniu GPS (ang. *Global Positioning System*), eksploracji otoczenia oraz elementach rywalizacji. Gry te wyróżniają się dokładnymi danymi lokalizacyjnymi i przedstawianiem ich w formie interaktywnej mapy, co pozwala na nowe doświadczenia w rozrywce i interakcji społecznej.

Jednym z najbardziej rozpoznawalnych tytułów jest GeoGuessr, który znacząco zyskał na popularności dopiero siedem lat po premierze, czyli podczas pandemii COVID-19 w 2020 roku. Gra pozwala na eksplorowanie świata bez wychodzenia z domu - zadaniem gracza jest poprawne odgadnięcie lokalizacji przedstawionego miejsca w ograniczonym czasie, zaznaczając swoją odpowiedź na interaktywnej mapie. Popularność GeoGuessr w tym okresie pokazuje, jak pandemia zmieniła nawyki konsumenckie i zwiększyła zainteresowanie grami wymagającymi zarówno spostrzegawczości, jak i wiedzy geograficznej.



Rysunek 3.5: Gra Geoguessr wydana w 2020 roku.

Innym przykładem jest mobilna gra Pokémon GO, stworzona przez firmę Niantic, Inc. i wydana w 2016 roku. Gra opiera się na technologii AR (ang. *Augmented Reality*) i pozwala graczom eksplorować swoje otoczenie w poszukiwaniu Pokémonów, łączyć wirtualny świat z rzeczywistym oraz wchodzić w interakcje z innymi graczami przy specjalnych punktach, takich jak PokéStopy czy Gyms. Pokémon GO jest częścią szeroko znanej franczyzy Pokémon, co znacząco przyczyniło się do popularyzacji gry i zwiększenia jej zasięgu, przyciągając zarówno fanów marki, jak i nowych graczy zainteresowanych nowatorską mechaniką AR.

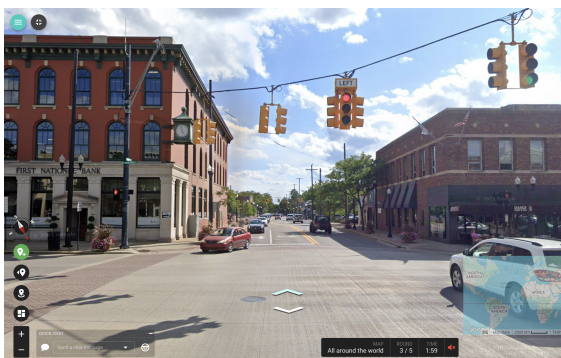


Rysunek 3.6: Gra PokemonGO wydana w 2016 roku.

3.3. Rodzaje gier konkurencyjnych

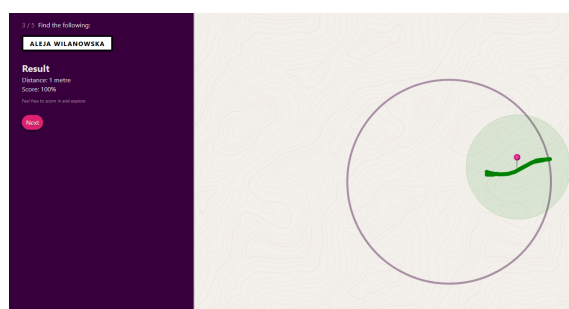
W przypadku rozpatrywania rodzaju gier konkurencyjnych, można ponownie wymienić wcześniej wspomnianego Geoguessra, ale również gry takie jak Geotastic czy Back of Your Hand.

Geotastic to gra bardzo zbliżona pod względem mechaniki do GeoGuessr. To, co przyciągnęło miliony graczy, to fakt, że jest darmową alternatywą, oferującą innowacyjne tryby rozgrywki oraz opartą na finansowaniu społecznościowym. Zaangażowana społeczność graczy aktywnie uczestniczy w rozwoju gry, dbając o jej aktualizację i utrzymanie, co dodatkowo wzmacnia jej atrakcyjność i trwałość w dłuższym okresie.



Rysunek 3.7: Gra Geotastic wydana w 2020 roku.

Z kolei Back of Your Hand skupia się na znajomości własnej okolicy i charakterystycznych obiektów. Gracz otrzymuje jedynie nazwę ulicy lub konkretnego miejsca, a następnie w określonym obszarze (np. w promieniu 1500 metrów) musi odnaleźć je na mapie. Im bliżej poprawnej lokalizacji gracz umieści swój „strzał”, tym więcej punktów zdobywa. Po zakończeniu rundy wyświetlane jest podsumowanie, w którym wszystkie miejsca i trafienia gracza prezentowane są na jednej interaktywnej mapie, umożliwiając łatwe przeanalizowanie wyników i strategii.



Rysunek 3.8: Gra Back of Your Hand wydana w 2020 roku.

4. WYKORZYSTANE TECHNOLOGIE

4.1. Frontendowe (Filip Jezierski)

Do implementacji frontendu zastosowano bibliotekę **React.js** z **TypeScriptem**, umożliwiającą tworzenie aplikacji w technologii **Progressive Web App**. Komunikację z backendem przez REST API zapewnia biblioteka **Axios**. Do obsługi map i lokalizacji użyto biblioteki **Leaflet**.

4.1.1. Progressive Web App

PWA (ang. *Progressive Web App*) to podejście do tworzenia aplikacji webowych, które umożliwia działanie podobne do natywnej aplikacji mobilnej lub desktopowej. PWA można zarówno używać w przeglądarce, jak i zainstalować jako niezależną od przeglądarki aplikację na urządzeniu. Technologia ta zapewnia też działanie podstawowych funkcji aplikacji bez dostępu do internetu.

Publikowanie PWA przez platformy dystrybucji cyfrowej takie jak Apple App Store czy Google Play jest możliwe, lecz opcjonalna - użytkownik może również zainstalować aplikację bezpośrednio z poziomu przeglądarki internetowej, wybierając opcję "Dodaj do ekranu głównego".

Aplikacja webowa musi spełniać poniżej wymienione wymagania aby mogła być zainstalowana na urządzeniu jako PWA:

- **Bezpieczne źródło** - aplikacja musi być serwowana przez HTTPS (ang. *Hypertext Transfer Protocol Secure*), aby zapewnić bezpieczeństwo i autentyczność danych.
- **Service Worker** - dzięki zastosowaniu tego mechanizmu aplikacja może ładować do pamięci podręcznej fragmenty, które jeszcze nie zostały użyte, co sprawia że można ją później uruchomić w trybie offline.
- **Plik manifestu** - w aplikacji powinien być zawarty plik JSON (ang. *JavaScript Object Notation*) zawierający następujące informacje: `name` lub `short_name`, zestaw ikon (w szczególności w rozdzielczościach 192 px i 512 px), `start_url` oraz parametr `display` (z jedną z wartości: `fullscreen`, `standalone`, `minimal-ui` lub `window-controls-overlay`).

4.1.2. React

React to popularna biblioteka JavaScript stworzona przez Facebooka, służąca do budowy nowoczesnych interfejsów użytkownika w oparciu o komponenty. Dzięki wykorzystaniu wirtualnego DOM-u (ang. *Document Object Model*) React umożliwia szybkie i wydajne aktualizacje widoku bez potrzeby bezpośrednich manipulacji w strukturze HTML.

Architektura komponentowa pozwala na dzielenie aplikacji na niezależne, łatwo testowalne i ponownie wykorzystywane elementy. W projekcie wykorzystano komponenty funkcyjne oraz hooki (`useState`, `useEffect`) do zarządzania stanem i cyklem życia komponentów.

React zapewnia dużą elastyczność oraz możliwość integracji z innymi bibliotekami frontendowymi. Do komunikacji z backendem wykorzystano funkcje asynchroniczne zdefiniowane w specjalnie wydzielonych serwisach. Dodatkowo, React umożliwia tworzenie aplikacji responsywnych, co było istotne w kontekście zapewnienia poprawnego działania na różnych typach urządzeń.

W projekcie zdecydowaliśmy się również na użycie języka TypeScript - nadzbioru JavaScriptu, który wprowadza statyczne typowanie. TypeScript ułatwia wczesne wykrywanie błędów, usprawnia podpowie-

dzi edytora oraz poprawia czytelność i bezpieczeństwo kodu. Dodatkowo umożliwia łatwe modelowanie danych przesyłanych między komponentami oraz poprzez API.

4.1.3. *Axios*

Axios to biblioteka JavaScript służąca do wykonywania asynchronicznych zapytań HTTP. W projekcie została użyta do komunikacji pomiędzy frontendem i backendem poprzez REST API.

W porównaniu do natywnej funkcji `fetch`, Axios oferuje uproszczoną składnię, automatyczne przetwarzanie odpowiedzi w formacie JSON oraz bardziej zaawansowane możliwości konfiguracji — takie jak ustawianie domyślnych nagłówków, bazowego URL, timeoutów i łatwiejsze przechwytywanie błędów. W połączeniu z TypeScriptem, Axios umożliwia również bardziej precyzyjne typowanie odpowiedzi serwera.

4.1.4. *Leaflet*

Leaflet to otwartoźródłowa biblioteka JavaScript, służąca do tworzenia interaktywnych map. W projekcie użyliśmy jej w połączeniu z React Leaflet - wrapperem, który udostępnia funkcjonalność Leafleta w postaci gotowych komponentów React.

Biblioteka Leaflet umożliwia wyświetlanie map z zewnętrznych źródeł (np. OpenStreetMap) oraz nanoszenie na nie dodatkowych elementów, takich jak znaczniki, warstwy czy lokalizacja użytkownika z wykorzystaniem geolokalizacji przeglądarki. Dodatkowo umożliwia wykrywanie interakcji z mapą, np. kliknięć, oraz zwracanie odpowiadających im współrzędnych geograficznych.

4.2. *Backendowe (Michał Turski)*

W backendzie zastosowano język **C#** z frameworkiem **.NET** oraz paczki NuGet w celu stworzenia aplikacji serwerowej realizującej wymagania biznesowe i obsługującej komunikację z bazą danych **PostgreSQL**.

4.2.1. *C# ASP.NET Core*

ASP.NET Core od lat utrzymuje się w czołówce najchętniej wybieranych frameworków do tworzenia aplikacji webowych i API. Jego popularność wynika przede wszystkim z wysokiej modularności oraz niezwyklej elastyczności w adaptowaniu się do różnych scenariuszy projektowych.

Atutem ASP.NET Core jest jego pełna wieloplatformowość — aplikacje mogą być uruchamiane na systemach Windows, Linux oraz macOS bez konieczności wprowadzania zmian w kodzie. To znacząco ułatwia wdrażanie aplikacji w różnorodnych środowiskach, w tym także w kontenerach Docker czy chmurze. Aktualnie w trakcie procesu wytwarzania oprogramowania wykorzystywany jest system Windows, WSL, a także aplikacja odpalana jest w kontenerze Dockerowym.

Ponadto środowisko to w znacznym stopniu wspiera rozwój oprogramowania zgodnie z zasadami SOLID. Dzięki wbudowanemu mechanizmowi wstrzykiwania zależności (dependency injection), ułatwione jest tworzenie spójnych i jednoznacznie zdefiniowanych interfejsów, a także separacja odpowiedzialności pomiędzy warstwami aplikacji.

4.2.2. *Wykorzystane pakiety NuGet*

W projekcie wykorzystano szereg pakietów NuGet, które usprawniają implementację kluczowych funkcjonalności oraz wspierają dobre praktyki projektowe. Pakiety NuGet to zewnętrzne biblioteki, które

można łatwo zintegrować z projektem .NET w celu rozszerzenia jego możliwości. Umożliwiają one ponowne wykorzystanie sprawdzonego kodu oraz automatyzację zarządzania zależnościami, co znacząco przyspiesza proces tworzenia i utrzymania aplikacji.

Wykorzystano następujące pakiety NuGet w aplikacji wraz z krótkim opisem:

- **AutoMapper** – Ułatwia przekształcanie danych pomiędzy klasami, co jest szczególnie przydatne przy mapowaniu obiektów domenowych na klasy DTO (ang. *Data Transfer Object*) oraz odwrotnie. Redukuje potrzebę ręcznego kopiowania właściwości, co przekłada się na większą czytelność i spójność kodu.
- **FluentValidation** – Usprawnia proces walidacji danych, wprowadzając dedykowaną warstwę odpowiedzialną za sprawdzanie poprawności obiektów wejściowych. Pozwala na definiowanie reguł walidacyjnych w sposób deklaratywny, oddzielając je od logiki aplikacji.
- **Microsoft.EntityFrameworkCore** – Biblioteka ORM umożliwiająca odwzorowywanie klas C# na struktury relacyjnej bazy danych. Umożliwia pracę z bazą danych z użyciem LINQ (ang. *Language Integrated Query*) oraz migracji, eliminując potrzebę pisania zapytań SQL (ang. *Structured Query Language*).
- **Npgsql.EntityFrameworkCore.PostgreSQL** – Dostarcza integrację Entity Framework Core z bazą danych PostgreSQL, umożliwiając pełne wsparcie dla tej platformy w środowisku .NET.
- **Swashbuckle.AspNetCore.Swagger** – Narzędzie generujące dokumentację API w formacie OpenAPI (Swagger). Umożliwia wygodne testowanie endpointów poprzez interaktywny interfejs webowy oraz automatyczne tworzenie opisów metod HTTP.
- **Microsoft.AspNetCore.Authentication.JwtBearer** – Zapewnia obsługę uwierzytelniania z użyciem tokenów JWT (ang. *JSON Web Token*), które są powszechnym rozwiązaniem przy autoryzacji użytkowników w aplikacjach webowych typu REST API.
- **xUnit** – Framework służący do tworzenia testów jednostkowych, umożliwiający automatyczne sprawdzanie poprawności działania kodu aplikacji.
- **Moq** – Biblioteka wspierająca testowanie poprzez łatwe tworzenie atrap (mocków) i symulowanie zachowań obiektów, co pozwala izolować testowane elementy od zależności.
-
- **MediatR** - Biblioteka implementująca wzorzec Mediator, który ułatwia komunikację między komponentami aplikacji poprzez pośrednika. Pomaga uporządkować przepływ komunikacji i usunąć bezpośrednie zależności między warstwami lub klasami.

4.3. Baza danych (Michał Turski)

W projekcie zastosowano relacyjną bazę danych **PostgreSQL**, która jest darmowym i otwartym oprogramowaniem typu **Open Source**. Jest to szczególnie korzystne w przypadku projektów niekomercyjnych.

Baza danych wyróżnia się zgodnością ze standardem SQL oraz obsługą zaawansowanych typów danych, takich jak **JSONB**, umożliwiając efektywne łączenie struktur relacyjnych z nierelacyjnymi.

Do zarządzania bazą danych wykorzystano narzędzia administracyjne, takie jak **pgAdmin** i **DB-aver**, oferujące graficzne interfejsy ułatwiające pracę z bazą danych.

5. NARZĘDZIA

5.1. System kontroli wersji (Michał Pawiłojć)

W trakcie realizacji projektu *UniGuesser* kluczową rolę odegrało zastosowanie systemu kontroli wersji Git oraz platformy GitHub. Ich wykorzystanie umożliwiło efektywne zarządzanie kodem źródłowym, koordynację pracy zespołu oraz utrzymanie przejrzystości i historii zmian w projekcie.

Git to rozproszony system kontroli wersji VCS (ang. *Version Control System*), stworzony przez Linusa Torvaldsa w 2005 roku. Jego podstawowym zadaniem jest śledzenie zmian w plikach projektu, umożliwienie pracy wielu programistów nad tym samym kodem oraz szybkie cofanie się do wcześniejszych wersji w razie potrzeby. Git opiera się na repozytorium lokalnym i zdalnym, co zapewnia dużą elastyczność pracy – możliwa jest nawet praca offline z późniejszą synchronizacją.

Do podstawowych operacji należą:

- `git init` – inicjalizacja nowego repozytorium,
- `git add` – dodanie zmian do indeksu (staging area),
- `git commit` – zapisanie zmian w historii projektu,
- `git branch` – zarządzanie gałęziami (np. tworzenie odgałęzień na nowe funkcjonalności),
- `git merge` – scalanie gałęzi,
- `git log` – przegląd historii commitów.

Stosowanie Gita pozwala na łatwe śledzenie błędów, eksperymentowanie z nowymi funkcjonalnościami bez wpływu na główny kod oraz pracę równoległą wielu osób. W przypadku zmian powodujących błąd powrót do działającej wersji jest praktycznie bezkosztowy.

Natomiast GitHub to jedna z najpopularniejszych platform umożliwiających przechowywanie zdalnych repozytoriów Git. Oferuje dodatkowe funkcje, takie jak:

- pull requesty (prośby o połączenie zmian z główną gałęzią projektu),
- code review (wzajemna kontrola kodu w zespole),
- zarządzanie zgłoszeniami (issues),
- integracja z narzędziami CI/CD (ang. *Continuous Integration/Continuous Delivery lub Deployment*) jak GitHub Actions.

W projekcie *UniGuesser* GitHub pełnił rolę centralnego punktu współpracy, umożliwiając m.in. zgłaszanie błędów, przeglądanie zmian oraz automatyczne uruchamianie testów jednostkowych i wdrożeń (CI/CD).

Repozytorium projektu zostało zorganizowane w sposób modułowy – z podziałem na frontend, backend i konfigurację Docker. Każdy członek zespołu pracował na oddzielnych branchach, a zmiany były scalane po przejściu procesu *code review*. Takie podejście zapewniło stabilność projektu i ułatwiło jego rozwój.

5.2. Konteneryzacja - Docker (Michał Pawiłojć)

Współczesne aplikacje webowe, takie jak *UniGuesser*, składają się z wielu komponentów - frontend, backend, baza danych - muszą one współdziałać w spójnym środowisku. Tradycyjna metoda instalowa-

nia zależności lokalnie prowadzi do problemów ze zgodnością, trudnością odtworzenia środowiska oraz błędami zależnymi od platformy. Rozwiązaniem tego problemu jest konteneryzacja – technika, która pozwala zbudować przenośne, spójne środowisko dla każdego komponentu aplikacji.

5.2.1. Konteneryzacja

Konteneryzacja to proces pakowania aplikacji wraz z całym jej środowiskiem uruchomieniowym (zależnościami, bibliotekami, konfiguracją, itd.) do jednego, samodzielnego pakietu zwanego **kontenerem**. Taki kontener może być uruchomiony na dowolnym systemie operacyjnym, który obsługuje kontenery, bez potrzeby rekonfiguracji lub instalacji dodatkowego oprogramowania.

Kontenery są lekką alternatywą dla maszyn wirtualnych. W odróżnieniu od VM-ek, nie zawierają pełnego systemu operacyjnego, a współdzielą jądro systemu hosta. Dzięki temu zużywają mniej zasobów, szybciej się uruchamiają i łatwiej się nimi zarządza.

Dzięki konteneryzacji możliwe jest podejście „*build once, run anywhere*” – czyli raz zbudowany obraz aplikacji może działać tak samo na komputerze deweloperskim, serwerze testowym i w środowisku produkcyjnym.

5.2.2. Docker

W naszym projekcie wykorzystaliśmy **Dockera** — najpopularniejsze narzędzie do konteneryzacji. Docker umożliwia budowanie własnych obrazów aplikacji za pomocą plików konfiguracyjnych `Dockerfile`, a także zarządzanie kontenerami lokalnie lub w chmurze.

5.2.3. Docker Compose

Projekt *UniGuesser* składa się z kilku współdziałających usług:

- Frontend (React),
- Backend (.NET),
- Baza danych (PostgreSQL).

Do zarządzania tymi usługami wykorzystano **Docker Compose**. Umożliwia ono deklaratywne opisanie całego środowiska w jednym pliku YAML. W ten sposób tworzona jest wewnętrzna sieć, w której poszczególne usługi są w stanie się komunikować.

Całe środowisko można uruchomić jedną komendą:

```
1 docker-compose up --build
```

5.2.4. Korzyści wynikające z konteneryzacji

Użycie Dockera i Composa przyniosło wiele korzyści:

- **Powtarzalność środowiska** — każdy członek zespołu uruchamiał projekt dokładnie w taki sam sposób.
- **Izolacja komponentów** — każda usługa działała w swoim kontenerze.
- **Łatwe wdrażanie** — obrazy Dockera działały tak samo lokalnie, jak i na produkcji.
- **Integracja z CI/CD** — obrazy mogły być budowane i testowane w GitHub Actions.

5.2.5. Ograniczenia

Choć konteneryzacja znacząco uprościła zarządzanie środowiskiem, zostały również napotkane pewne trudności:

- **Krzywa uczenia** — wymagana była znajomość Dockera i Composa.
- **Wydajność na Windowsie** — niektóre operacje były wolniejsze w WSL2.
- **Debugowanie** — błędy konfiguracyjne były trudniejsze do wychwycenia.

Pomimo tych wyzwań, konteneryzacja odegrała kluczową rolę w sukcesie projektu.

5.3. Zarządzanie projektem - YouTrack (Karina Wołoszyn)

Do zarządzania projektem wykorzystany został Youtrack, który jest narzędziem typu ITS (ang. *Issue Tracker System*) stworzonym przez firmę JetBrains. Opiera się on na technologii Ajax i wspiera szeroki zakres metodyk zarządzania projektem, w tym podejść zwinnych, które również zostało wykonane do realizacji tego projektu.

System Youtrack umożliwia m.in. planowanie i organizację zadań, przypisywanie ich do konkretnych członków, ustalanie priorytetów, śledzenia postępów prac, dodawanie tagów, załączników czy tworzenie raportów. Dzięki rozbudowanym możliwościom filtrowania, szacowania i określania ilości spędzonego czasu, personalizacji widoków możliwe było bieżące monitorowanie i reagowanie na pojawiające się ewentualne problemy.

W ramach projektu przyjęto podejście oparte na zmodyfikowanej wersji metodyki Kanban, dostosowanej do potrzeb zespołu i charakteru realizowanego zadania. Tablica Kanban została podzielona na sześć głównych etapów:

- **Backlog** - przeznaczony do przechowywania wszystkich potrzebnych zadań do zrealizowania projektu. Te zadania obejmują duży zakres obowiązków, które są w następnym etapie rozdzielane na mniejsze, bardziej precyzyjne zadania, jeżeli są wybrane do realizacji w najbliższym sprincie.
- **Backlog sprintu** - zbierane są tu zadania do bieżącego sprintu. Dokładnie jest określany zakres wymagań, który musi spełnić, aby w pełni je zrealizować.
- **Wykonanie** - aktualnie wykonywane zadania. Przydzielone są one do konkretnego członka zespołu i mają ustawiony priorytet oraz liczbę ustalonych story points'ów.
- **Testowanie** - zbierane są tutaj wykonane zadania oczekujące na przetestowanie przez innego członka zespołu albo osobę wykonującą dane zadanie. W przypadku błędów lub niespełnienia wymagań, zadanie zostaje zwrócone do poprzedniego etapu w celu wprowadzenia poprawek.
- **Recenzja** - wykonane zadania oczekują na recenzję przez innego członka zespołu niż ten, który go wykonywał. Pozwala to na lepszą identyfikację błędów lub potencjalnych błędów. Podobnie jak w przypadku testowania, w przypadku niespełnienia kryteriów zadanie wraca do etapu 'Wykonanie'.
- **Zrobione** - przeznaczony do przechowywania wykonanych, przetestowanych i zrecenzowanych zadań.

5.4. Zintegrowane środowisko programistyczne - VSC, VS, Rider (Michał Pawłójć)

W trakcie realizacji projektu wykorzystano różne zintegrowane środowiska programistyczne IDE, dostosowane do technologii używanych w poszczególnych komponentach aplikacji. Odpowiedni dobór

narzędzi znacząco przyczynił się do efektywnej pracy zespołu, ułatwiając zarówno pisanie kodu, jak i jego debugowanie, testowanie oraz integrację z systemem kontroli wersji Git.

5.4.1. Backend: Visual Studio i Rider

Głównym środowiskiem wykorzystywanym do pracy nad backendem był **Visual Studio**, w wersji Community 2022, z uwagi na jego doskonałe wsparcie dla platformy .NET i języka C#.

Visual Studio oferowało pełny zestaw funkcjonalności ułatwiających pracę z aplikacjami ASP.NET Core, m.in.:

- integrację z systemem projektów .NET,
- intuicyjne debugowanie,
- wizualizację struktur danych,
- automatyczne wykrywanie problemów w kodzie (analiza statyczna),
- możliwość tworzenia i uruchamiania testów jednostkowych,
- integrację z narzędziami do konteneryzacji, np. Docker Tools.

Niektórzy członkowie zespołu korzystali również z alternatywnego IDE: **JetBrains Rider**. Był on preferowany w szczególności przez osoby przyzwyczajone do narzędzi JetBrains oraz pracujące głównie w środowisku Linux lub WSL. Rider zapewniał szybkie działanie, inteligentne podpowiedzi (ang. *code completion*) i integrację z systemami bazodanowymi oraz Dockerem.

Oba środowiska dobrze współpracowały z systemem kontroli wersji Git oraz pozwalały na łatwą konfigurację środowisk uruchomieniowych (launch profiles), co było kluczowe przy testowaniu aplikacji w kontenerach.

5.4.2. Frontend: Visual Studio Code

Do pracy nad frontendem, który został zaimplementowany w technologii **React** z wykorzystaniem **TypeScript**, wykorzystywano głównie **Visual Studio Code** (VS Code). Jest to lekkie, ale bardzo elastyczne narzędzie, które doskonale sprawdza się w projektach opartych na JavaScript, dzięki bogatemu ekosystemowi rozszerzeń.

VS Code ułatwiał szybkie uruchamianie aplikacji frontendowej za pomocą skryptów `npm`, a także pozwalał na debugowanie kodu przeglądarkowego poprzez wbudowany debugger.

5.4.3. Wspólne praktyki i integracja IDE z projektem

Pomimo stosowania różnych środowisk, zespół utrzymywał spójne praktyki pracy:

- **Standaryzacja formatowania kodu** — z wykorzystaniem plików konfiguracyjnych (`.editorconfig`, `.prettierrc`),
- **Wspólne launch profile** — umożliwiające uruchamianie backendu lokalnie lub w kontenerze,
- **Integracja z GitHubem** — dzięki której możliwa była praca zdalna, pull requesty i code review bezpośrednio z poziomu IDE,
- **Debugowanie lokalne i w kontenerze** — możliwe dzięki odpowiednim rozszerzeniom i konfiguracji,
- **Wsparcie dla Docker Compose** — uruchamianie całego środowiska aplikacji z poziomu IDE.

Różnorodność wykorzystywanych środowisk nie stanowiła problemu dzięki modularnej architekturze projektu oraz konteneryzacji, która gwarantowała, że kod backendu i frontend działał w identycznym środowisku niezależnie od lokalnych ustawień IDE.

Zastosowanie odpowiednich IDE w zależności od warstwy aplikacji pozwoliło zwiększyć produktywność zespołu. Visual Studio oraz Rider znakomicie sprawdziły się przy rozwoju backendu w .NET, natomiast VS Code (ang. *Visual Studio Code*) był naturalnym wyborem do pracy nad frontendem w React. Elastyczność i rozszerzalność tych środowisk umożliwiła sprawną pracę zespołową oraz efektywne wdrażanie zmian w całym cyklu życia aplikacji.

5.5. Projektowanie interfejsu użytkownika - Figma (Karina Wołoszyn)

W celu zbudowania wspólnej i klarownej wizji interfejsu użytkownika, została wykorzystana Figma, czyli aplikacja webowa pozwalająca na projektowanie prototypów UI/UX (ang. *User Interface/User Experience*). Narzędzie to umożliwia pracę zespołową w czasie rzeczywistym, czyniąc komunikację i wprowadzanie szybszą, i prostszą. W projekcie *UniGuesser* szczególny nacisk położono na minimalizm i prostotę zaprojektowanego interfejsu. Jednocześnie uwzględniono Księgę Identyfikacji Wizualnej Politechniki Gdańskiej, aby właściwie odwzorować kolorystykę oraz rozmieszczenie elementów zgodnie z przyjętymi standardami. Figma posiada szeroki wachlarz funkcjonalności, które w dużym stopniu usprawniają pracę zespołu. Do najważniejszych z nich należą:

- **Zarządzanie warstwami i grupami** - pozwala na precyzyjną organizację elementów, co jest szczególnie ważne przy tworzeniu skomplikowanej aplikacji, aby zachować jej spójność wizualną.
- **Tworzenie komponentów i wariantów** - umożliwia tworzenie wielu wariantów wykorzystywanych elementów UI, co pozwala na testowanie różnych rozwiązań bez konieczności zmiany kodu.
- **Integracje z narzędziami deweloperskimi** - wspierają płynne przejście od fazy projektowej do wdrożenia. Jednym z takich narzędzi jest GitHub, który oferuje łatwą konfigurację, możliwość dodawania zgłoszeń oraz pull requestów, wspomagając kontrolę wersji całego projektu.
- **Eksport zasobów i generowanie fragmentów kodu CSS** - przyspiesza implementację zaprojektowanych widoków.
- **Biblioteki stylów i zasobów** - pozwala na utrzymanie spójności wizualnej przechowując zestaw fontów, kolorów, efektów i rozmiarów elementów dostępnych dla całego zespołu.

Dzięki tym zaawansowanym funkcjom Figma stała się kluczowym elementem w realizacji procesu projektowania interfejsu użytkownika *UniGuesser*.

6. ANALIZA WYMAGAŃ

6.1. *Koncepcja gry (Karina Wołoszyn)*

UniGuesser to interaktywna aplikacja edukacyjno-rozrywkowa, której celem jest odgadywanie lokalizacji związanych z kampusem Politechniki Gdańskiej. Gra łączy elementy zabawy, edukacji oraz eksploracji przestrzeni uczelni, a dzięki technologii PWA użytkownicy mogą korzystać z niej na komputerach i urządzeniach mobilnych bez konieczności instalacji.

Celem UniGuesser jest:

- angażowanie graczy w interaktywną zabawę polegającą na identyfikacji lokalizacji na podstawie zdjęć,
- rozwijanie orientacji przestrzennej w obrębie kampusu uczelni,
- aktywne poznawanie nowych miejsc i życia akademickiego w formie przyjemnej i edukacyjnej.

Aplikacja jest skierowana do:

- studentów Politechniki Gdańskiej, w tym nowych studentów poznających kampus,
- pracowników uczelni,
- osób odwiedzających kampus, np. podczas dni otwartych,
- entuzjastów gier geolokalizacyjnych i edukacyjnych, którzy cenią naukę przez zabawę.

UniGuesser może pełnić funkcję:

- narzędzia integracyjnego dla studentów i pracowników uczelni,
- elementu wydarzeń akademickich, takich jak dni otwarte czy spotkania organizacyjne,
- gry terenowej zachęcającej do aktywnego zwiedzania kampusu.

Podczas rozgrywki gracz otrzymuje losowo wybrane zdjęcie przedstawiające charakterystyczną lokalizację. Zadaniem użytkownika jest zaznaczenie jej na interaktywnej mapie. Po zatwierdzeniu odpowiedzi system:

- oblicza odległość między wskazanym a faktycznym miejscem,
- przyznaje punkty proporcjonalnie do dokładności,
- pokazuje prawidłowe położenie lokalizacji na mapie.

Zaletami aplikacji UniGuesser są między innymi:

- dostępność na różnych urządzeniach dzięki technologii PWA,
- połączenie zabawy z nauką i poznawaniem kampusu,
- motywacja do rywalizacji poprzez system punktacji i rankingów,
- zachęta do aktywnego odkrywania otoczenia w trybie terenowym,
- wsparcie integracji społeczności akademickiej i współzawodnictwa między graczami.

6.2. *Przebieg gry (Karina Wołoszyn)*

Rozgrywka w UniGuesser przebiega w kilku etapach, które zapewniają płynność gry i umożliwiają graczowi śledzenie swoich wyników:

1. **Rozpoczęcie gry** - użytkownik wybiera tryb i trudność rozgrywki, a w przypadku gdy nie jest zalogowany - wpisuje swój nick. System umożliwia także kontynuację poprzedniej gry wraz z zapisanymi punktami i historią odpowiedzi lub rozpoczęcie nowej sesji.
2. **Losowanie lokacji** - aplikacja losowo wybiera zdjęcie lokalizacji z bazy danych wraz z jej współrzędnymi, przy czym w danej sesji nie powtarzają się już wyświetlone miejsca.
3. **Odgadywanie lokalizacji** - gracz wskazuje na interaktywnej mapie miejsce odpowiadające wylosowanemu zdjęciu. Interfejs pozwala przybliżyć i oddalać mapę oraz przesunąć ją w celu precyzyjnego wyboru. Gracz może wielokrotnie zaznaczać punkty, jednak decyduje ostatnie kliknięcie.
4. **Zatwierdzenie wyboru** - gracz potwierdza swój wybór, klikając przycisk zatwierdzający decyzję.
5. **Wyświetlenie wyniku** - po zatwierdzeniu odpowiedzi system oblicza odległość między wskazanym a faktycznym miejscem, przyznaje odpowiednią liczbę punktów i pokazuje prawidłowe położenie lokalizacji na mapie.
6. **Podsumowanie rundy** - gracz otrzymuje informacje o zdobytych punktach w danej rundzie oraz może przeglądać krótką statystykę swoich postępów.
7. **Kontynuacja lub zakończenie gry** - jeśli gracz nie rozegrał jeszcze wszystkich rund, system pozwala przejść do kolejnej. W przeciwnym wypadku wyświetlane jest podsumowanie całej gry w formie przejrzystej tabeli z możliwością podglądu wyników każdej rundy.

Taka struktura rozgrywki sprawia, że gra jest czytelna, intuicyjna i pozwala zarówno na zabawę, jak i naukę poprzez aktywne poznawanie kampusu Politechniki Gdańskiej.

6.3. Tryby rozgrywki (Karina Wołoszyn)

Gra oferuje dwa tryby rozgrywki, które urozmaicają zgadywanie lokalizacji. Oprócz trybu podstawowego, rozgrywanego w miejscu, dostępny jest również tryb terenowy, przeznaczony do gry na zewnątrz i w ruchu. Poniżej przedstawiono szczegółowy opis trybu terenowego.

Tryb terenowy pobiera lokalizację gracza za pomocą sygnału GPS dostępnego w urządzeniu przenośnym i wyświetla ją na mapie. Gracz może w prosty sposób zmieniać widok na ekranie, przełączając się między mapą a zdjęciem. Tryb terenowy posiada również system podpowiedzi oparty na mechanice „ciepło-zimno”, mający na celu ograniczenie potencjalnej frustracji gracza. Podpowiedź może być użyta maksymalnie trzy razy i działa poprzez wyświetlanie odpowiednich kolorów: czerwonego, jeśli gracz zbliża się do celu w ciągu ostatnich 30 sekund, lub niebieskiego, jeśli się oddala. Gdy gracz znajduje się w promieniu 5 metrów od miejsca docelowego i zatwierdzi swój „strzał”, system kończy rundę i wyświetla wynik tak jak w trybie podstawowym.

W trybie terenowym kluczowa jest dokładność użytkownika. Poprzez aktywne poruszanie się i poszukiwanie miejsca na podstawie jego widoku, gracz w praktyczny sposób poznaje teren kampusu politechniki oraz lokalizacje związane z uczelnią.

6.4. Wymagania funkcjonalne (Karina Wołoszyn)

Wymagania funkcjonalne określają kluczowe funkcje, które aplikacja musi realizować. Definiują one, jak system powinien się zachowywać w określonych sytuacjach, jakie działania ma wykonywać oraz jakie rezultaty ma osiągać w odpowiedzi na określone dane wejściowe.

Fundamentalnym wymogiem całego projektu było sprecyzowanie wymagań dotyczących podstawowej rozgrywki. To właśnie w tym trybie gracz spędza najwięcej czasu, dlatego kluczowe jest, aby była ona dopracowana, angażująca i zachęcająca do ponownego grania. W związku z tym wyróżniono następujące wymagania:

- **Adekwatna punktacja** - aby gracz czuł się odpowiednio nagrodzony lub ukarany za swoją odpowiedź, system powinien przydzielać punkty proporcjonalnie do odległości pomiędzy faktycznym miejscem a wskazaniem gracza. Aplikacja musi zatem nie tylko obliczać tę odległość, lecz także przeliczać ją na odpowiednią liczbę punktów.
- **Zapisywanie wyników pojedynczego gracza** - wyniki każdej rozgrywki są zapisywane, aby gracz mógł w przyszłości przeglądać swoje wcześniejsze sesje. Podgląd wyników umożliwia analizę i rozwój umiejętności, a także pozwala zobaczyć, gdzie dokładnie padły odpowiedzi na mapie wraz z informacjami o lokalizacjach.
- **Tryby gry** - rozgrywka posiada dwa tryby gry: klasyczny (losowe lokacje z bazy danych) oraz tematyczny (lokacje powiązane z wybranym obszarem, np. kampusem uczelni lub konkretnym miastem).
- **Rejestracja i logowanie** - system powinien rozróżniać role użytkowników poprzez proces rejestracji i logowania. Zwykły użytkownik ma ograniczone uprawnienia w porównaniu z Moderatorem czy Administratorem. Nowe konto można założyć za pomocą prostego formularza rejestracyjnego.
- **Dodawanie nowych lokacji przez użytkowników** - zarejestrowani użytkownicy mogą zgłaszać nowe lokacje do bazy danych, dodając opis, współrzędne geograficzne oraz zdjęcie. Propozycje trafiają do weryfikacji przed publikacją.
- **Nadzór lokacji** - role takie jak Administrator czy Moderator mają możliwość zatwierdzania, edytowania lub odrzucania dodanych lokalizacji, aby zapobiec nadużyciom, naruszeniom praw autorskich oraz publikacji niepożądanych treści.
- **Ranking graczy** - ranking zwiększa element rywalizacji między graczami i wspiera rozwój społeczności. Przedstawia on nick gracza, jego wynik, tryb rozgrywki oraz poziom trudności. Ranking posiada paginację i ograniczenie do jednego rekordu dla danego zestawu ustawień, co usprawnia jego działanie.
- **Prosty i intuicyjny interfejs** - użytkownik powinien już podczas pierwszego kontaktu z systemem intuicyjnie rozumieć jego działanie. Projekt interfejsu opiera się na minimalizmie i spójności wizualnej w całej aplikacji.
- **Kolorystyka interfejsu** - zastosowana kolorystyka powinna być zgodna z barwami Politechniki Gdańskiej, zachowując estetykę i czytelność.
- **Lokacje związane z życiem uczelnianym** - baza danych powinna zawierać lokacje związane z Politechniką Gdańską i życiem studenckim, np. budynki uczelni, akademiki, kawiarnie, parki czy znane miejsca spotkań studentów.
- **Responsywne UI** - interfejs powinien poprawnie działać na komputerach oraz urządzeniach mobilnych, dostosowując układ elementów do rozmiaru ekranu, aby zachować komfort gry niezależnie od urządzenia.
- **Responsywna mapa** - mapa powinna umożliwiać graczowi umieszczanie pinezek jako odpowiedzi, a także przybliżanie, oddalanie, przesuwanie i automatyczne wyśrodkowanie na docelową lokację po zakończeniu rundy.

- **Aplikacja PWA** - aplikacja powinna wspierać instalację w formie PWA, umożliwiając korzystanie z niej offline i z poziomu ekranu głównego urządzenia.
- **Kontynuacja gry** - system powinien pozwalać użytkownikowi na kontynuację niedokończonych rozgrywek z zachowaniem dotychczasowych punktów oraz historii odpowiedzi.
- **Hamburger menu** - przy małym rozmiarze ekranu opcje z górnego paska menu powinny być zwijane do tzw. hamburger menu, przy zachowaniu kolejności i logiki nawigacji.
- **System podpowiedzi „ciepło-zimno”** - w trybie terenowym gracz ma dostęp do systemu podpowiedzi, który pomaga mu zlokalizować docelowe miejsce, nie ujawniając go wprost.

Wymagania te stanowią podstawę do stworzenia aplikacji spełniającej główne założenia projektu, a jednocześnie podnoszącej jakość doświadczenia i satysfakcję użytkownika.

6.5. Wymagania niefunkcjonalne (Filip Jezierski)

Wymagania niefunkcjonalne określają kryteria jakościowe, które aplikacja powinna spełniać, aby zapewnić odpowiednie doświadczenie użytkownika oraz efektywne działanie systemu. W kontekście opracowywanej aplikacji mobilnej, kluczowe wymagania niefunkcjonalne obejmują aspekty takie jak wydajność, bezpieczeństwo oraz użyteczność.

Wymagania jakościowe

- obsługa różnych rozdzielczości ekranów i urządzeń mobilnych,
- dostępność aplikacji na popularnych platformach mobilnych (Android, iOS), a także jako aplikacja webowa,
- intuicyjny i responsywny interfejs użytkownika,
- aplikacja powinna w odpowiedni sposób obsługiwać błędy i wyjątki, takie jak nieprawidłowe dane wejściowe czy brak internetu, zapewniając stabilność działania,
- ochrona danych użytkowników poprzez implementację odpowiednich mechanizmów bezpieczeństwa - szyfrowanie danych, bezpieczne logowanie, ochrona przed atakami typu SQL Injection i XSS.

Wymagania wydajnościowe

- szybkie (<2 s) ładowanie aplikacji i minimalny czas oczekiwania na reakcję interfejsu użytkownika,
- ładowanie zdjęć lokalizacji w czasie nie dłuższym niż 3 sekundy,
- szybkie (<10 s) ładowanie danych geolokalizacyjnych zapewniające płynne działanie trybu gry opartego na lokalizacji,

6.6. Wybór platformy (Filip Jezierski)

Wybór odpowiedniej platformy dla opracowywanej aplikacji mobilnej jest kluczowym elementem procesu projektowego, mającym istotny wpływ na jej funkcjonalność, dostępność oraz doświadczenie użytkownika, a także łatwość wdrożenia i utrzymania. Na etapie analizy wymagań podjęto decyzję o stworzeniu dwóch testowych wersji aplikacji: aplikacji webowej typu PWA (Progressive Web App) oraz natywnej aplikacji mobilnej wykorzystującej technologię .NET MAUI (Multi-platform App UI). Obie platformy oferują unikalne korzyści i wyzwania, które zostały dokładnie rozważone w kontekście specyfikacji projektu.

6.6.1. Natywna aplikacja mobilna

Aplikacje natywne są tworzone z wykorzystaniem dedykowanych narzędzi i języków programowania przeznaczonych dla konkretnych systemów operacyjnych, takich jak Android czy iOS. W przypadku technologii .NET MAUI możliwe jest tworzenie aplikacji wieloplatformowych przy zachowaniu wspólnej bazy kodu, co znacząco skraca czas implementacji i ułatwia utrzymanie.

Jednakże proces publikacji aplikacji natywnych w sklepach takich jak Google Play czy Apple App Store wiąże się z dodatkowymi wymaganiami i procedurami zatwierdzania, a także z koniecznością budowania i testowania aplikacji na różne systemy operacyjne, co może wydłużyć czas wdrożenia.

6.6.2. Aplikacja webowa PWA

Aplikacja webowa typu PWA (Progressive Web App) łączy cechy klasycznej strony internetowej z funkcjonalnością aplikacji mobilnej. Dzięki wykorzystaniu nowoczesnych technologii webowych, możliwe jest stworzenie aplikacji działającej w trybie offline, instalowanej bezpośrednio z przeglądarki oraz zapewniającej wrażenia zbliżone do aplikacji natywnej.

Ze względu na łatwość dostępu (poprzez przeglądarkę internetową) oraz brak konieczności przechodzenia przez proces zatwierdzania w sklepach aplikacji, PWA stanowi atrakcyjną opcję dla naszego projektu. Jednakże, pewne ograniczenia związane z dostępem do funkcji urządzenia (takich jak geolokalizacja) oraz wydajnością w porównaniu do aplikacji natywnych muszą być uwzględnione podczas projektowania i implementacji.

6.7. Ograniczenia projektu (Filip Jezierski)

Ograniczenia projektu wynikają przede wszystkim z przyjętych założeń technologicznych oraz charakteru opracowywanego rozwiązania. Kluczowe znaczenie mają tu specyfika aplikacji webowej typu PWA, zastosowanie mechanizmów geolokalizacji jako podstawowego elementu funkcjonalnego, a także decyzje dotyczące sposobu pozyskiwania i wykorzystania materiałów wizualnych. Wymienione czynniki w naturalny sposób wpływają na zakres możliwości implementacyjnych, wydajność rozwiązania oraz doświadczenie użytkownika końcowego.

6.7.1. Środowisko aplikacji

Aplikacja będzie udostępniana jako aplikacja webowa, dostępna przez przeglądarkę internetową, z możliwością instalacji na ekranie głównym urządzenia. Funkcjonalność trybu gry wykorzystującego geolokalizację będzie dostępna wyłącznie na urządzeniach mobilnych, które wspierają tę technologię. W przypadku urządzeń stacjonarnych, nawet jeśli geolokalizacja jest technicznie możliwa, jej dokładność jest niewystarczająca do zapewnienia poprawnego działania tej funkcji.

6.7.2. Zdjęcia lokalizacji

W projekcie zrezygnowano z wykorzystywania zdjęć 360° pochodzących z zewnętrznych źródeł, takich jak interfejsy API udostępniane przez firmy trzecie. Decyzję tę podjęto z uwagi na chęć uniknięcia potencjalnych problemów związanych z licencjonowaniem, ograniczoną dostępnością materiałów oraz zróżnicowaną jakością zdjęć. Dodatkowo korzystanie z takich źródeł mogłoby wprowadzać ograniczenia dotyczące liczby i różnorodności dostępnych lokalizacji.

W celu uproszczenia procesu pozyskiwania materiałów wizualnych oraz umożliwienia użytkownikom współtworzenia bazy lokalizacji, aplikacja będzie wykorzystywać standardowe zdjęcia dwuwymiarowe.

Tego typu fotografie nie wymagają specjalistycznego sprzętu i mogą być wykonywane przy użyciu dowolnego smartfona. Użytkownicy będą mieli możliwość przesyłania własnych zdjęć lokalizacji, które po weryfikacji przez moderatorów zostaną dodane do bazy danych aplikacji.

Takie rozwiązanie zapewnia pełną kontrolę nad jakością i zgodnością materiałów z założeniami projektu, a jednocześnie pozwala na stopniowe rozszerzanie bazy lokalizacji o nowe zdjęcia, dostarczane zarówno przez zespół projektowy, jak i społeczność użytkowników.

6.8. Kryteria akceptacji (Filip Jezierski)

Ogólne kryteria:

- Aplikacja działa na urządzeniach mobilnych z systemami Android i iOS, na komputerach stacjonarnych oraz w przeglądarkach internetowych
- Intuicyjny interfejs użytkownika umożliwiający łatwe korzystanie z aplikacji
- Responsywny design dostosowujący się do różnych rozmiarów ekranów, w szczególności urządzeń mobilnych
- Stabilne działanie aplikacji bez krytycznych błędów i awarii
- Testy manualne wszystkich zaplanowanych funkcjonalności przeprowadzone na wszystkich wspieranych platformach

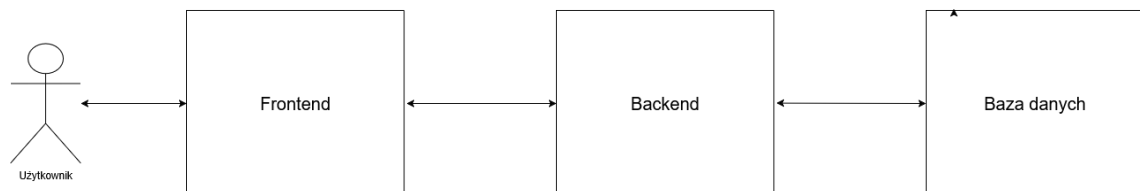
Zaimplementowane funkcjonalności:

- Rejestracja i logowanie użytkowników
- Wybór trybu rozgrywki i poziomu trudności
- Dwa tryby gry:
 - odgadywanie lokalizacji na podstawie zdjęcia poprzez wskazanie jej na mapie
 - odnajdywanie lokalizacji na podstawie zdjęcia i potwierdzanie obecności poprzez geolokalizację
- Możliwość przerywania i kontynuowania rozgrywki
- Wyświetlanie podsumowania wyników każdej rundy gry
- Historia rozgrywek dostępna w profilu użytkownika
- Ogólnodostępny ranking graczy
- Dodawanie lokalizacji przez użytkowników (po weryfikacji przez moderatorów)

7. PROJEKT SYSTEMU (MICHAŁ TURSKI I FILIP JEZERSKI)

Niniejszy rozdział przedstawia szczegółowy projekt systemu aplikacji webowej **UniGuesser**. Opisane zostaną kluczowe elementy architektury systemu, w tym warstwy aplikacji, baza danych oraz najbardziej kluczowe interakcje użytkownika z systemem. Zaczynając od ogólnego przeglądu architektury, przejdziemy do szczegółowego omówienia poszczególnych komponentów i ich wzajemnych relacji.

W aplikacji zastosowano architekturę warstwową, której szczegółowa implementacja zostanie opisana w niniejszym rozdziale. Na najwyższym poziomie abstrakcji system można podzielić na trzy główne warstwy: warstwę prezentacji (frontend), warstwę logiki biznesowej (backend) oraz warstwę persystencji danych (baza danych). Podział ten odzwierciedla sposób izolowania odpowiedzialności poszczególnych komponentów. Dzięki temu użytkownik komunikuje się z aplikacją poprzez interfejs użytkownika, który następnie przekazuje żądania do warstwy logiki biznesowej. Ta z kolei przetwarza żądania, wykonuje odpowiednie operacje i komunikuje się z bazą danych w celu przechowywania lub pobierania danych.



Rysunek 7.1: Uproszczony schemat architektury systemu UniGuesser – ogólny podział na warstwy

7.1. Architektura logiki biznesowej (Michał Turski)

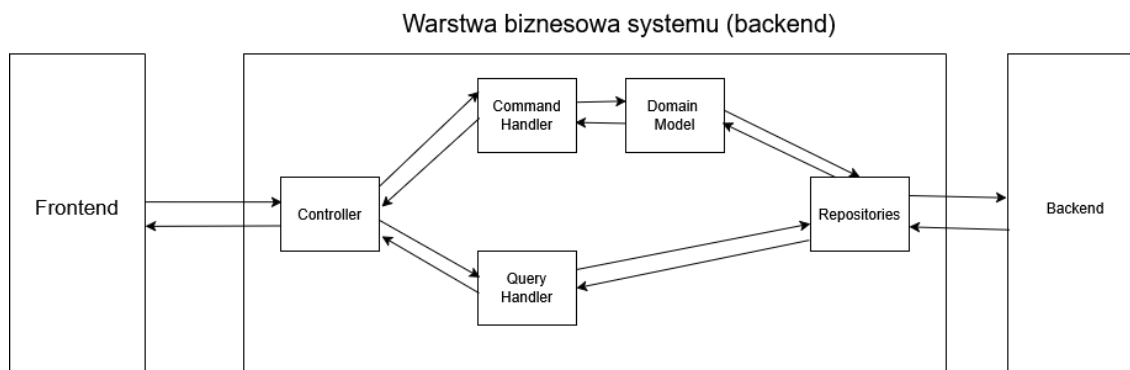
Architektura w aplikacji **UniGuesser** została zbudowana w oparciu o CQRS (ang. *Command Query Responsibility Segregation*) oraz DDD (ang. *Domain-Driven Design*), które są obecnie standardem w wytwarzaniu oprogramowania. [dodać źródło na potwierdzenie słów] Ponadto zdecydowaliśmy na wybór architektury monolitycznej ze względu na szybkość rozwoju i łatwość wdrożenia, co pomogło skupić się nam na aspekcie funkcjonalnym aplikacji w początkowej fazie jej tworzenia.

7.1.1. Wzorzec CQRS

Wykorzystany został również wzorzec CQRS. Zakłada on rozdzielenie operacji modyfikujących stan systemu (komendy) od operacji, które odczytują dane (zapytania). Pozwala to na lepsze skalowanie i rozdzielenie logiki aplikacji. Dzięki temu można optymalizować każdą z tych operacji niezależnie, co przekłada się na wydajność i czytelność kodu. W kontekście skalowalności systemu zastosowano uproszczoną implementację wzorca CQRS z pojedynczą bazą danych, która jest wystarczająca przy przewidywanym obciążeniu nieprzekraczającym 50 zapytań na sekundę. Taka architektura zapewnia odpowiednią wydajność przy jednoczesnym zachowaniu prostoty infrastruktury.

7.1.2. Zastosowanie DDD

Model systemu został zaprojektowany zgodnie z podejściem DDD, które kładzie nacisk na zrozumienie i modelowanie domeny problemowej. W aplikacji definiujemy trzy główne warstwy DDD: warstwę domeny, warstwę aplikacji oraz warstwę infrastruktury. Każda z warstw znajduje się w osobnym module,



Rysunek 7.2: Schemat komunikacji przy zastosowaniu wzorca CQRS

co ułatwia zarządzanie kodem i jego rozwój. Dodatkowo w aplikacji wydzielony został czwarty moduł „API”, który odpowiada za komunikację z warstwą prezentacji, co zmniejsza złożoność warstwy aplikacji. Moduł „API” obsługuje zarówno zapytania, jak i komendy, delegując je odpowiednio do warstwy aplikacji, która następnie komunikuje się z warstwą domeny i infrastruktury w celu realizacji żądanych operacji.

7.2. Baza danych (Michał Turski)

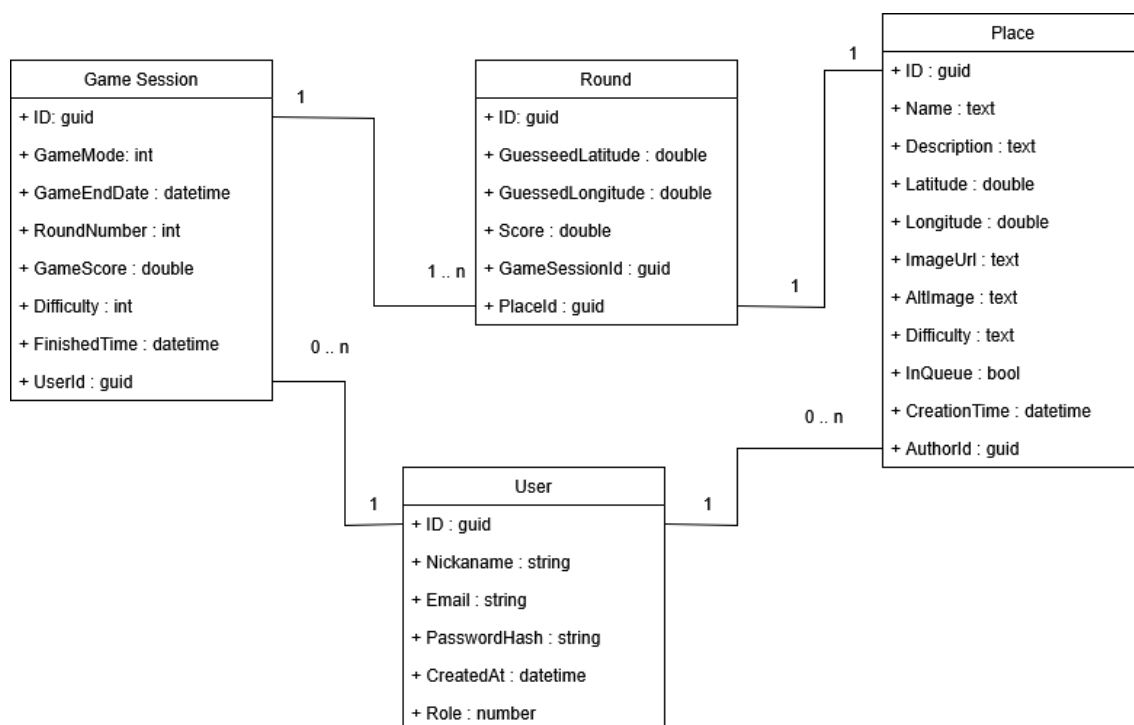
W aplikacji zastosowano relacyjną bazę danych, do przechowywania informacji o aktualnych i przeszłych sesjach gier, użytkownikach, lokalizacjach oraz wynikach. Zdecydowaliśmy się na wykorzystanie relacyjnej bazy danych ze względu na dużą ilość powiązań między encjami oraz potrzebę zapewnienia integralności danych między nimi. Relacyjne bazy danych zapewniają ACID (Atomicity, Consistency, Isolation, Durability), co jest kluczowe dla zachowania spójności danych w rozgrywce.

7.2.1. Diagram ERD

Przedstawiony na rysunku 7.3 diagram ERD (ang. *Entity-Relationship Diagram*) ilustruje strukturę bazy danych aplikacji UniGuesser, pokazując główne tabele oraz relacje między nimi. Użytkownik może posiadać wiele sesji gier, a każda sesja ma określoną liczbę rund, z których każda posiada przypisaną do siebie lokację. Ponadto każde miejsce może mieć swojego autora (użytkownika), który dodał je do bazy danych.

7.2.2. Opis głównych tabel i związków

- **Users (Użytkownicy)** – tabela przechowująca informacje o użytkownikach aplikacji.



Rysunek 7.3: Diagram ERD dla bazy danych aplikacji UniGuesser

Tabela 7.1: Tabela Users

Atrybut	Opis	Typ / Uwagi
ID	Unikalny identyfikator użytkownika.	Klucz główny
Nickname	Unikalna nazwa użytkownika.	Tekst
Email	Unikalny adres e-mail użytkownika.	Tekst
PasswordHash	Zaszyfrowane hasło użytkownika.	Tekst
CreatedAt	Data i czas utworzenia konta.	Data/Czas
Role	Rola użytkownika w systemie (np. użytkownik, administrator).	Enumeracja

GameSessions (Sesje gier) – tabela przechowująca informacje o aktywnych i zakończonych rozgrywkach.

Tabela 7.2: Tabela GameSessions

Atrybut	Opis	Typ / Uwagi
ID	Unikalny identyfikator sesji gry.	Klucz główny
GameMode	Tryb gry w danej sesji.	Enumeracja
GameEndDate	Data, do której rozgrywka jest aktywna.	Data/Czas
RoundNumber	Aktualnie rozgrywana runda.	Liczba
GameScore	Łączna liczba punktów zdobytych.	Liczba całkowita
Difficulty	Poziom trudności sesji.	Enumeracja
FinishedTime	Data zakończenia rozgrywki.	Data/Czas
UserID	Identyfikator użytkownika.	Klucz obcy → Users

Rounds (Rundy) – tabela przechowująca informacje o poszczególnych rundach w sesji gry.

Tabela 7.3: Tabela Rounds

Atrybut	Opis	Typ / Uwagi
ID	Unikalny identyfikator rundy.	Klucz główny
GuessedLatitude	Szerokość geograficzna wskazana przez użytkownika.	Liczba zmiennie-przecinkowa
GuessedLongitude	Długość geograficzna wskazana przez użytkownika.	Liczba zmiennie-przecinkowa
Score	Punkty zdobyte w rundzie.	Liczba całkowita
GameSessionID	Identyfikator sesji gry.	Klucz obcy → GameSessions
LocationID	Identyfikator lokacji.	Klucz obcy → Places

Places (Lokacje) – tabela przechowująca informacje o lokacjach dostępnych w grze.

Tabela 7.4: Tabela Places

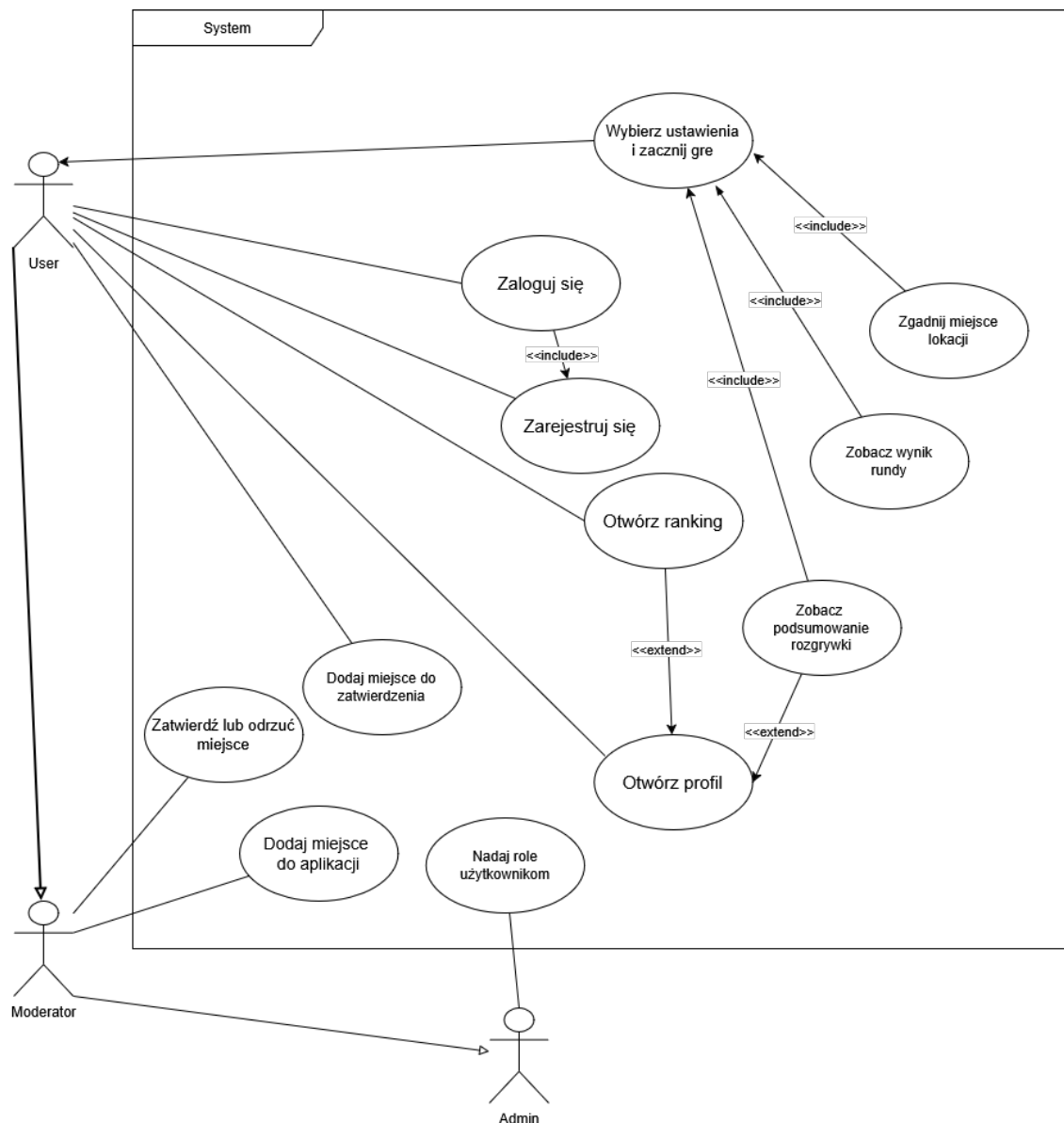
Atrybut	Opis	Typ / Uwagi
ID	Unikalny identyfikator lokacji.	Klucz główny
Name	Nazwa lokacji.	Tekst
Description	Opis lokacji.	Tekst
Latitude	Szerokość geograficzna lokacji.	Liczba zmiennie- przecinkowa
Longitude	Długość geograficzna lokacji.	Liczba zmiennie- przecinkowa
Difficulty	Poziom trudności lokacji.	Enumeracja
ImageUrl	Adres URL obrazu lokacji.	Tekst
AltImage	Tekst alternatywny dla obrazu lokacji.	Tekst
CreationTime	Data i czas dodania lokacji.	Data/Czas
AuthorID	Identyfikator autora lokacji.	Klucz obcy → Users

7.3. Analiza interakcji użytkownika z systemem (Michał Turski)

W rozdziale przedstawione zostaną możliwości i przebieg kluczowych funkcjonalności aplikacji UniGuesser z perspektywy użytkownika końcowego. Opisane zostaną główne przypadki użycia oraz interakcje między użytkownikiem a systemem podczas typowej rozgrywki.

7.3.1. Przypadki użycia aplikacji

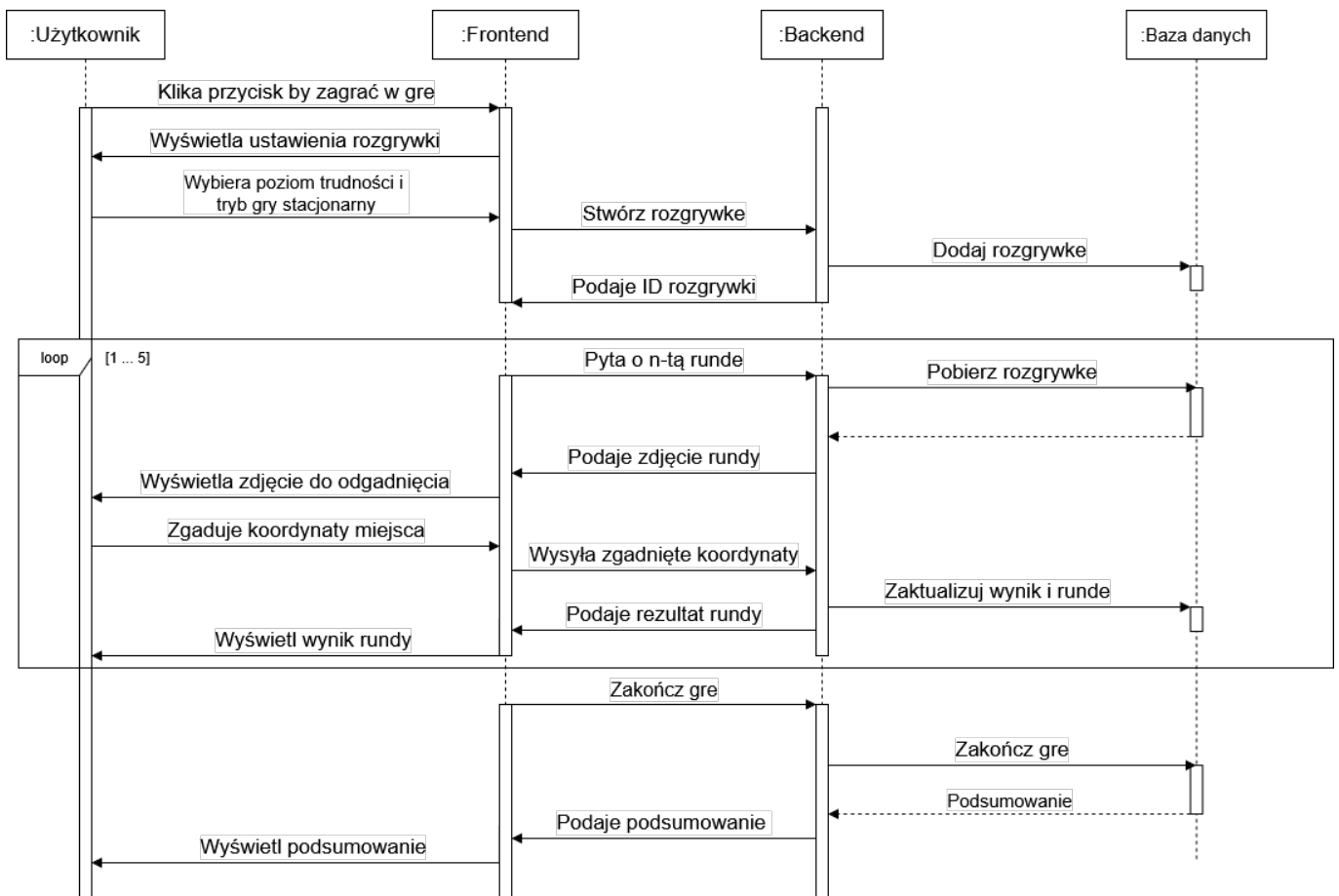
Przedstawiony na rysunku 7.4 diagram przypadków użycia ilustruje główne funkcjonalności aplikacji UniGuesser z perspektywy użytkownika moderatora i administratora. Użytkownik może rejestrować się i logować do systemu, rozpoczynać nowe sesje gier, uczestniczyć w rozgrywkach, przeglądać wyniki innych użytkowników, oraz proponować nowe lokacje. Moderator dodatkowo może zatwierdzać lub odrzucać proponowane lokacje, natomiast administrator ma uprawnienia do zarządzania użytkownikami.



Rysunek 7.4: Diagram przypadków użycia aplikacji UniGuesser

7.3.2. Diagram sekwencji rozgrywki

Diagram sekwencji przedstawiony na rysunku 7.5 ilustruje przebieg interakcji między użytkownikiem a systemem podczas typowej rozgrywki w aplikacji UniGuesser. Proces rozpoczyna się od momentu, gdy użytkownik uruchamia nową grę, poprzez kolejne rundy, aż do momentu zakończenia sesji i wyświetlenia wyników końcowych.



Rysunek 7.5: Diagram sekwencji przedstawiający przebieg rozgrywki w aplikacji UniGuesser

7.4. Estetyka i design (Filip Jezierski)

Wygląd aplikacji został zaprojektowany z myślą o prostocie, intuicyjności oraz atrakcyjności wizualnej. Poniżej przedstawiono kluczowe elementy designu interfejsu użytkownika.

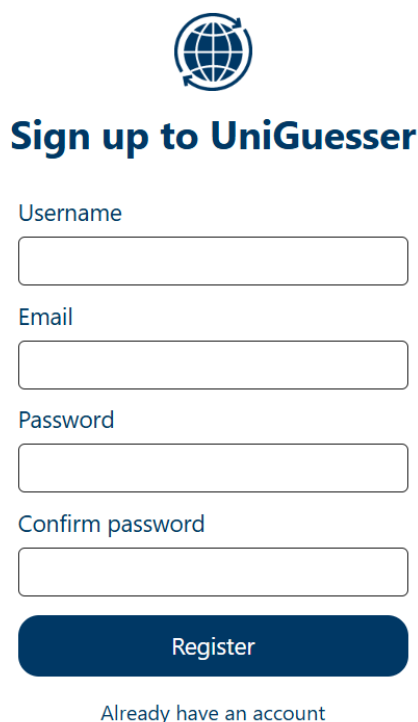
7.4.1. Ogólny projekt interfejsu

Design interfejsu użytkownika naszej aplikacji skupia się na minimalizmie i czytelności. Stosowane są przyjęte jako standardowe układy elementów, takie jak menu nawigacji w górnej części ekranu, co ułatwia poruszanie się po aplikacji, nawet dla nowych użytkowników. Podstrony są zaprojektowane tak, aby unikać nadmiaru informacji, co pozwala użytkownikom skupić się na kluczowych funkcjonalnościach. W trakcie rozgrywki centralnym elementem interfejsu jest interaktywna mapa, która umożliwia użytkownikom łatwe wskazywanie lokalizacji, a pozostałe elementy interfejsu są rozmieszczone w sposób intuicyjny wokół niej.

7.4.2. Kolorystyka

Aplikacja wykorzystuje dwukolorową paletę barw, opartą na odcieniach białego i ciemnoniebieskiego, inspirowaną kolorystyką Politechniki Gdańskiej. Biały kolor dominuje w tle, co zapewnia czystość i przejrzystość interfejsu, natomiast niebieskie akcenty są używane do wyróżnienia kluczowych elementów, takich jak przyciski, nagłówki czy linki. Przykładowe zastosowanie palety kolorów przedstawia rysunek 7.6. Taka kolorystyka zapewnia odpowiedni kontrast i ułatwia czytelność tekstu, zwłaszcza przy ko-

rzystaniu z aplikacji w różnych warunkach oświetleniowych, co jest szczególnie istotne ze względu na terenowy charakter gry.



The image shows a registration form for the UniGuesser application. At the top center is a logo consisting of a globe with a circular arrow around it. Below the logo is the title "Sign up to UniGuesser" in a bold, dark blue font. The form contains four input fields, each with a label above it: "Username", "Email", "Password", and "Confirm password". All labels and the "Register" button text are in a dark blue font. The "Register" button is a solid dark blue rectangle. Below the button is a link that says "Already have an account" in a smaller, lighter blue font.

Rysunek 7.6: Przykładowy ekran rejestracji użytkownika w aplikacji UniGuesser, ilustrujący zastosowaną kolorystykę

7.4.3. Typografia

Aplikacja wykorzystuje czcionkę Roboto, która jest nowoczesna, czytelna i estetyczna. Czcionka ta jest szeroko stosowana w projektach webowych ze względu na swoją uniwersalność i dobrą czytelność na ekranach o różnych rozdzielczościach.

7.4.4. Ikony

W interfejsie użytkownika wykorzystano zestaw prostych i intuicyjnych ikon, pochodzących z biblioteki Google Icons. Ikony te są spójne stylistycznie i pomagają użytkownikom szybko zidentyfikować funkcje oraz akcje dostępne w aplikacji. Dzięki zastosowaniu powszechnie rozpoznawalnych symboli, nawigacja staje się bardziej intuicyjna, co przekłada się na lepsze doświadczenie użytkownika.

7.4.5. Responsywność

Aplikacja została zaprojektowana z myślą o responsywności, co oznacza, że dostosowuje się do różnych rozmiarów ekranów i urządzeń. Dzięki temu użytkownicy mogą korzystać z aplikacji zarówno na komputerach stacjonarnych, jak i urządzeniach mobilnych, bez utraty funkcjonalności czy estetyki. Elementy interfejsu, takie jak przyciski, formularze czy mapy, automatycznie dostosowują swoje rozmiary i układ w zależności od dostępnej przestrzeni ekranowej. Na mniejszych ekranach, takich jak smartfony, menu nawigacyjne jest ukryte za ikoną hamburgera, co pozwala zaoszczędzić miejsce i utrzymać przejrzystość interfejsu. Ponadto, kluczowe funkcje pozostają łatwo dostępne, a tekst i grafiki są skalowane

odpowiednio do rozdzielczości ekranu, zapewniając optymalne doświadczenie użytkownika niezależnie od urządzenia.

8. IMPLEMENTACJA

8.1. Implementacja wyglądu i funkcjonalności frontendu

W tej sekcji omówione zostaną kluczowe aspekty implementacji frontendu aplikacji, w tym architektura, struktura UI i komponentów, warstwa wizualna, obsługa interakcji użytkownika, integracja z geolokalizacją, implementacja trybów gry, komunikacja z backendem oraz mechanizmy PWA.

8.1.1. *Architektura frontendu (Karina Wołoszyn)*

8.1.2. *Struktura UI i komponentów (Karina Wołoszyn)*

8.1.3. *Warstwa wizualna (Karina Wołoszyn)*

8.1.4. *Obsługa interakcji użytkownika (Karina Wołoszyn)*

8.1.5. *Integracja z geolokalizacją (Filip Jezierski)*

Do implementacji funkcjonalności związanej z lokalizacją użytkownika wykorzystano wbudowane w przeglądarki Geolocation API. Mechanizm ten działa tylko w kontekście bezpiecznym (HTTPS / localhost) i wymaga jawnej zgody użytkownika. W aplikacji wykorzystano zarówno jednorazowe pobieranie pozycji (`getCurrentPosition`), jak i ciągłe śledzenie (`watchPosition`) dla trybów gry, które wymagają aktualizacji współrzędnych w czasie rzeczywistym.

Główne założenia implementacyjne:

- Sprawdzenie uprawnień przez Permissions API przed żądaniem lokalizacji, by poprawnie obsłużyć przypadki odrzuconej zgody.
- Użycie opcji `enableHighAccuracy`, `timeout` i `maximumAge` do balansowania dokładności i zużycia baterii.
- Obsługa błędów (brak zgody, timeout, niedostępność) oraz mechanizm fallback — ręczne wprowadzenie lokalizacji przez użytkownika.
- Wysyłanie zaktualizowanej pozycji na backend przez bezpieczne wywołanie API (z autoryzacją JWT) oraz aktualizacja widoku mapy (np. Leaflet / Mapbox).
- W trybie PWA uwzględniono, że żądania lokalizacji mogą być ograniczone w tle — należy testować zachowanie na docelowych platformach.

Poniżej znajduje się przykładowy fragment kodu (TypeScript) użyty we frontendzie, ilustrujący sposób integracji z Geolocation API:

```
1 const watchGeolocation = (): number | null => {
2   if (!('geolocation' in navigator)) {
3     console.error('Geolocation is not supported by your browser.')
4     return null
5   }
6   return navigator.geolocation.watchPosition(
7     (position) => {
8       const coords: Coordinates = {
9         latitude: position.coords.latitude,
```

```

10         longitude: position.coords.longitude,
11     }
12     selectLocation(coords)
13     setGeoError(null)
14 },
15 (error) => {
16     console.error('Unable to retrieve location. Please enable location
17         services.', error)
18     if (error.code === 1) {
19         setGeoError('Location access denied. Please allow location
20             access in your browser settings.')
21     } else if (error.code === 2) {
22         setGeoError('Location unavailable. Please check your GPS
23             settings.')
24     } else {
25         setGeoError('Unable to retrieve location. Please try again.')
26     }
27 },
28 {
29     enableHighAccuracy: true,
30     maximumAge: 3000,
31     timeout: 5000,
32 }
33 )
34 }

```

Listing 8.1: Przykład wykorzystania Geolocation API do śledzenia pozycji użytkownika

Po uzyskaniu zgody użytkownika na dostęp do lokalizacji, funkcja `watchGeolocation` rozpoczyna śledzenie pozycji. W przypadku każdej aktualizacji pozycji wywoływana jest funkcja `selectLocation`, która aktualizuje stan aplikacji oraz widok mapy. W przypadku błędów odpowiednie komunikaty są wyświetlane użytkownikowi, aby poinformować go o problemach z lokalizacją.

Poza rozgrywką, Geolocation API jest również wykorzystywane w formularzu dodawania nowych punktów gry, gdzie użytkownik może automatycznie wypełnić swoje współrzędne geograficzne. Aby ułatwić ponowne użycie kodu, stworzono dedykowany hook Reactowy `useGeolocation`, który zarządza stanem lokalizacji i błędów związanych z geolokalizacją:

```

1 export const useGeolocation = () => {
2     const [coordinates, setCoordinates] = useState<Coordinates | null>(null)
3     const [geolocationError, setError] = useState<string | null>(null)
4
5     useEffect(() => {
6         if (!navigator.geolocation) {
7             setError('Geolocation is not supported by your browser.')
8             return
9         }
10
11         readCoordinates()
12     }, [])
13 }

```

```

14   const readCoordinates = () => {
15       navigator.geolocation.getCurrentPosition(
16           (position) => {
17               setCoordinates({
18                   latitude: position.coords.latitude,
19                   longitude: position.coords.longitude,
20               })
21               setError(null)
22           },
23           (err) => {
24               if (err.code === 1) {
25                   setError('Location access denied. Please allow location access in your browser settings.')
26               } else if (err.code === 2) {
27                   setError('Location unavailable. Please check your GPS settings.')
28               } else if (err.code === 3) {
29                   setError('Location request timed out. Please try again.')
30               } else {
31                   setError(err.message)
32               }
33           },
34           {
35               enableHighAccuracy: true,
36               maximumAge: 0,
37               timeout: 5000,
38           }
39       )
40   }
41
42   return { coordinates, setCoordinates, readCoordinates, geolocationError }
43 }

```

Listing 8.2: Przykład użycia Geolocation API we własnym hooku

Powyższy kod definiuje hook `useGeolocation`, który umożliwia komponentom formularza łatwe pobieranie i zarządzanie współrzędnymi geograficznymi użytkownika. W przypadku błędów odpowiednie komunikaty są ustawiane w stanie, aby informować użytkownika o problemach z lokalizacją. Poniżej znajduje się przykład użycia tego hooka w komponencie formularza:

```

1  const AddPlace: React.FC = () => {
2      const { coordinates, setCoordinates, readCoordinates, geolocationError } =
          useGeolocation();
3
4      /* ... pozostała czesc komponentu ... */
5
6      return (
7          <div>
8              <h2>Add New Place</h2>
9              {geolocationError && <p className="error">{geolocationError}</p>}

```

```

10         <button onClick={readCoordinates}>Get Current Location</button>
11         {coordinates && (
12             <div>
13                 <p>Latitude: {coordinates.latitude}</p>
14                 <p>Longitude: {coordinates.longitude}</p>
15             </div>
16         )}
17         {/* Dodatkowe pola formularza dla nazwy miejsca, opisu itp. */}
18     </div>
19 );
20 };

```

Listing 8.3: Przykład użycia hooka useGeolocation w komponencie formularza

8.1.6. Komunikacja z backendem (Filip Jezierski)

Frontend komunikuje się z backendem za pomocą asynchronicznych wywołań API przy użyciu biblioteki Axios. Wszystkie żądania do chronionych zasobów zawierają token JWT w nagłówku autoryzacji, co umożliwia bezpieczną komunikację i weryfikację użytkownika po stronie serwera.

Komunikacja została zrealizowana przez warstwę serwisów (ang. services). Każdy serwis odpowiada za jedną grupę zasobów (np. AccountService, PlaceService) i korzysta ze współdzielonego klienta HTTP, co umożliwia centralne zarządzanie konfiguracją połączenia, taką jak adres bazowy API, nagłówki czy obsługa błędów.

Poniżej znajduje się przykładowy fragment kodu (TypeScript) przedstawiający implementację serwisu do zarządzania miejscami (PlaceService) z wykorzystaniem tokenu JWT do autoryzacji:

```

1  export class PlaceService {
2      private axiosInstance: AxiosInstance
3
4      constructor() {
5          this.axiosInstance = axios.create({
6              baseURL: PLACE_API_URL,
7              headers: {
8                  'Content-Type': 'application/json',
9              },
10         })
11     }
12
13     // Metoda pobierająca wszystkie miejsca w kolejce do sprawdzenia, dostępna
14     // tylko dla autoryzowanych użytkowników
15     async getAllPlacesInQueue(): Promise<ShowPlaceDto[]> {
16         const result = await this.axiosInstance.get<ShowPlaceDto[]>('/to_check',
17             {
18                 headers: {
19                     Authorization: `Bearer ${sessionStorage.getItem(
20                         ACCOUNT_TOKEN_KEY)} `,
21                 },
22             })
23         return result.data
24     }
25 }

```

```

21     }
22
23     /* ... pozostałe metody komunikacji z backendem ... */
24 }
25
26 export default new PlaceService()

```

Listing 8.4: Konfiguracja instancji Axios z tokenem JWT

Stała `PLACE_API_URL` definiuje bazowy adres API dla zasobów związanych z miejscami. W każdej metodzie serwisu, takiej jak `getAllPlacesInQueue`, token JWT jest pobierany z `sessionStorage` i dołączany do nagłówka `Authorization` w formacie `Bearer <token>`. Dzięki temu backend może zweryfikować tożsamość użytkownika i jego uprawnienia do dostępu do żądanych zasobów.

8.1.7. Mechanizmy PWA (Filip Jezierski)

Aplikacja została zaprojektowana jako Progressive Web Application (PWA), co umożliwia użytkownikom instalację aplikacji na urządzeniach mobilnych oraz korzystanie z niej w trybie offline. Kluczowe mechanizmy PWA wymagane do poprawnego działania aplikacji to:

- **Service Worker:** W bibliotece Vite wykorzystano wtyczkę `vite-plugin-pwa`, która automatycznie generuje i rejestruje Service Workera. Service Worker zarządza cache'owaniem zasobów statycznych (HTML, CSS, JavaScript, obrazy) oraz dynamicznych danych API, co pozwala na szybkie ładowanie aplikacji nawet przy braku połączenia z internetem.
- **Manifest aplikacji:** Plik manifestu definiuje metadane aplikacji, takie jak nazwa, ikony, kolor motywu oraz sposób uruchamiania. Dzięki temu użytkownicy mogą dodać aplikację do ekranu głównego swoich urządzeń mobilnych, co zapewnia doświadczenie podobne do natywnej aplikacji. Manifest został zdefiniowany w pliku konfiguracyjnym Vite, gdzie określono podstawowe informacje o aplikacji oraz ikony w różnych rozmiarach.
- **Responsywność:** Aplikacja została zaprojektowana z myślą o różnych rozmiarach ekranów, co zapewnia optymalne doświadczenie użytkownika zarówno na komputerach, jak i urządzeniach mobilnych.

Poniżej znajduje się użyta konfiguracja Vite, która zawiera zarówno rejestrację Service Workera, jak i definicję manifestu aplikacji:

```

1 export default defineConfig({
2   plugins: [
3     react(),
4     svgr(),
5     VitePWA({
6       registerType: 'autoUpdate',
7       includeAssets: ['favicon.ico', 'logo192.png', 'logo512.png'],
8       manifest: {
9         name: 'Uniguesser',
10        short_name: 'Uniguesser',
11        description: 'A fun and engaging guessing game',
12        theme_color: '#000000',
13        icons: [
14          {
15            src: 'logo192.png',

```

```

16         sizes: '192x192',
17         type: 'image/png',
18     },
19     {
20         src: 'logo512.png',
21         sizes: '512x512',
22         type: 'image/png',
23     },
24 ],
25 },
26     workbox: {
27         globPatterns: ['**/*.{js,css,html,ico,png,svg,json}'],
28         runtimeCaching: [
29             {
30                 urlPattern: /^https:\/\/.*\.tile\.openstreetmap\.org
31                     \/.*/i,
32                 handler: 'CacheFirst',
33                 options: {
34                     cacheName: 'openstreetmap-tiles',
35                     expiration: {
36                         maxEntries: 500,
37                         maxAgeSeconds: 60 * 60 * 24 * 30, // 30 days
38                     },
39                     cacheableResponse: {
40                         statuses: [0, 200],
41                     },
42                 },
43             },
44         ],
45     },
46 ],
47 // ... pozostała konfiguracja Vite ...
48 })

```

Listing 8.5: Konfiguracja Vite do obsługi PWA

Konfiguracja obejmuje definicję manifestu PWA, zawierającego podstawowe informacje o aplikacji, parametry wyświetlania oraz zestaw ikon wykorzystywanych podczas instalacji na urządzeniu. Dodatkowo zdefiniowano ustawienia service workera generowanego przez vite-plugin-pwa. Obejmują one cache'owanie plików statycznych, określone w sekcji globPatterns oraz zasobów mapowych pobieranych z OpenStreetMap, zdefiniowanych w sekcji runtimeCaching, co poprawia szybkość działania aplikacji i umożliwia jej podstawowe funkcjonowanie przy ograniczonym dostępie do sieci.

8.1.8. Implementacja rozgrywki (Filip Jezierski)

Implementacja logiki rozgrywki opiera się na dwóch głównych komponentach: GameSettings, odpowiedzialnym za konfigurację nowej sesji gry, oraz Game, który obsługuje właściwy przebieg rund.

W komponencie GameSettings realizowane jest zarządzanie ustawieniami gracza, obsługa trybu gościa i użytkownika zalogowanego, zapisywanie konfiguracji w sessionStorage, a także decyzja o

kontynuowaniu istniejącej gry lub rozpoczęciu nowej. Komunikacja z API odbywa się poprzez metody serwisu `gameService`, które pozwalają m.in. sprawdzić bieżący stan gry, utworzyć nową sesję lub porzucić trwającą rozgrywkę. W przypadku wykrycia aktywnej gry użytkownik otrzymuje możliwość podjęcia działania poprzez wyświetlany modal.

```
1 const saveGameSettings = (data: StartGameData) => {
2     window.sessionStorage.setItem(SELECTED_GAME_MODE, data.gameMode)
3     window.sessionStorage.setItem(SELECTED_DIFFICULTY_KEY, data.difficulty)
4
5     if (!accountService.isLoggedIn()) {
6         window.sessionStorage.setItem(USER_NICKNAME_KEY, data.nickname)
7     }
8 }
```

Listing 8.6: Metoda w komponencie `GameSettings` zapisująca ustawienia gry w `sessionStorage`

Główny komponent `Game` odpowiada za przebieg sesji gry. Po załadowaniu sprawdzana jest możliwość wznowienia gry poprzez istniejący token lub inicjalizowana jest nowa rozgrywka.

```
1 useEffect(() => {
2     const controller = new AbortController()
3     const signal = controller.signal
4
5     if (gameService.hasToken()) {
6         getGame(signal)
7     } else {
8         startGame(signal)
9     }
10
11     return () => {
12         controller.abort()
13     }
14 }, [])
```

Listing 8.7: Fragment kodu w komponencie `Game` odpowiedzialny za decyzję o wznowieniu lub rozpoczęciu nowej gry

Następnie, jeśli rozgrywka z poprzedniej sesji ma być kontynuowana, wywoływana jest odpowiednia runda gry, na podstawie danych uzyskanych z serwera.

```
1 const getGame = async (signal: AbortSignal) => {
2     setLoading(true)
3     setError(null)
4
5     try {
6         const response = await gameService.checkGameState(signal)
7         console.log('Game state fetched successfully:', response)
8
9         // Load game mode from session storage when resuming a game
10        const savedGameMode = window.sessionStorage.getItem(SELECTED_GAME_MODE)
11        as GameMode | null
12        if (savedGameMode) {
```



```

12         setGameMode(savedGameMode)
13     }
14
15     startRound(response.actualRoundNumber)
16 } catch (err: any) {
17     if (err.name === 'CanceledError') {
18         console.error('Request aborted by the abort controller (2nd fetch prevention)')
19         return
20     } else {
21         setError('Failed to fetch data. Please try again later.')
22         const nickname = window.sessionStorage.getItem(USER_NICKNAME_KEY)
23         const newController = new AbortController()
24         const newSignal = newController.signal
25         if (!nickname) {
26             throw new Error('User nickname is missing')
27         }
28         startGame(newSignal)
29     }
30 } finally {
31     setLoading(false)
32 }
33 }

```

Listing 8.8: Funkcja pobierająca stan gry w komponencie Game

W przeciwnym wypadku, tworzona jest nowa sesja na podstawie ustawień zapisanych w `sessionStorage` przez `GameSettings`.

```

1 const startGame = async (signal: AbortSignal) => {
2     setLoading(true)
3     setError(null)
4
5     try {
6         const nickname = window.sessionStorage.getItem(USER_NICKNAME_KEY)
7         const difficulty = window.sessionStorage.getItem(
8             SELECTED_DIFFICULTY_KEY) as Difficulty | null
9         const selectedGameMode = window.sessionStorage.getItem(
10             SELECTED_GAME_MODE) as GameMode | null
11         if (!difficulty) {
12             throw new Error('Difficulty not selected')
13         }
14         if (!selectedGameMode) {
15             throw new Error('Game mode not selected')
16         }
17         if (!nickname) {
18             throw new Error('User not logged in')
19         }
20         setGameMode(selectedGameMode)
21         await gameService.startNewGameSession(nickname, difficulty,
22             selectedGameMode, signal)
23     }
24 }

```

```

20     startRound(0)
21   } catch (err: any) {
22     if (err.name === 'CanceledError') {
23       console.error('Request aborted by the abort controller (2nd fetch prevention)')
24     } else {
25       setError('Failed to fetch data. Please try again later.')
26       console.error('Error fetching data:', err)
27     }
28   } finally {
29     setLoading(false)
30   }
31 }

```

Listing 8.9: Funkcja rozpoczynająca nową grę w komponencie Game

Komponent pobiera dane dotyczące miejsca do odgadnięcia, wyświetla je użytkownikowi oraz przyjmuje współrzędne wskazane przez gracza. Po potwierdzeniu wyboru wynik rundy jest walidowany przez API, które zwraca m.in. współrzędne właściwej lokalizacji, odległość trafienia oraz aktualny wynik.

Ze względu na podobieństwo logiki między trybami komponent Game obsługuje zarówno tryb klasyczny, jak i geolokalizacyjny. Wybór interfejsu jest dokonywany przez komponent, renderując odpowiedni wariant (ClassicGameInterface lub GeolocationGameInterface) w zależności od wybranego trybu. Każda runda kończy się przejściem do kolejnej lub, w przypadku ostatniej, zakończeniem gry i przekierowaniem użytkownika do ekranu wyników. State komponentu zarządza takimi elementami jak numer rundy, wczytany obraz, współrzędne celów, wynik punktowy oraz informacje o błędzie.

Komponenty interfejsu gry są odpowiedzialne za prezentację danych i interakcję z użytkownikiem. W trybie klasycznym użytkownik może wskazać lokalizację na mapie poprzez kliknięcie, natomiast w trybie geolokalizacyjnym aplikacja automatycznie pobiera aktualną pozycję użytkownika i umożliwia jej zatwierdzenie jako odpowiedź. Oba interfejsy obsługują wyświetlanie informacji o rundzie, takie jak numer rundy, aktualny wynik oraz komunikaty o błędach.

8.2. Implementacja backendu i logiki aplikacji

W tej sekcji omówione zostaną kluczowe aspekty implementacji backendu aplikacji, rozgrywki mechanizmy autoryzacji i uwierzytelniania użytkowników, zarządzanie danymi oraz obsługa błędów, a także problemy, które napotkano podczas implementacji.

8.2.1. Implementacja rozgrywki

Poniżej przedstawiono sposób implementacji przebiegu rozgrywki, zgodny z diagramem sekwencji pokazanym na rysunku 7.5.

Proces rozpoczęcia rozgrywki inicjuje wywołanie kontrolera `api/game/start`, który odpowiada za utworzenie nowej *Sesji Rozgrywki* (7.2). W ramach tego etapu backend generuje odpowiednią liczbę rund, przypisuje im losowe miejsca do zgadnięcia, nadaje sesji status `InProgress`, a następnie zwraca unikalny identyfikator gry w postaci GUID. W przypadku użytkownika niezalogowanego aplikacja dodatkowo generuje i zwraca tymczasowy token, który umożliwia jego uwierzytelnienie w kolejnych żądaniach.

Aby pobrać dane dotyczące danej rundy, frontend wywołuje endpoint `api/game/gameId/round/roundNumber`,

gdzie `gameId` jest identyfikatorem sesji, a `roundNumber` – numerem aktualnej rundy. Backend zwraca ścieżkę do obrazu przedstawiającego miejsce, które należy odgadnąć. Obraz może być przechowywany lokalnie na serwerze lub stanowić odnośnik do zewnętrznego źródła.

Odpowiedź użytkownika jest obsługiwana przez endpoint `api/game/gameId/round/roundNumber/check`. Aplikacja weryfikuje lokalizację podaną przez użytkownika, oblicza odległość pomiędzy faktycznym miejscem a wskazanym punktem oraz na tej podstawie przyznaje punkty za daną rundę.

Po zakończeniu wszystkich rund gracz wywołuje endpoint `api/game/gameId/finish`, który finalizuje rozgrywkę. Backend podsumowuje wyniki, aktualizuje status sesji na `Completed` i zwraca końcowy rezultat użytkownikowi. Dla użytkowników zalogowanych rezultat jest trwale zapisywany w bazie, natomiast dla użytkowników niezalogowanych sesja jest automatycznie usuwana po upływie jednej godziny.

8.2.2. Autoryzacja i uwierzytelnianie użytkowników

Generowanie i struktura tokenu JWT służącego do autoryzacji użytkowników

W celu autoryzacji wykorzystany został JSON Web Token. Jest to standard wymiany informacji między dwoma stronami w formie obiektu JSON, który jest bezpiecznie podpisany cyfrowo.

Składa się on z trzech niezależnych części zakodowanych w formacie Base64: nagłówek (header), ładunku (payload) oraz podpisu (signature). Nagłówek zawiera informacje o algorytmie podpisu i typie tokenu, ładunek zawiera dane użytkownika i inne informacje, a podpis służy do weryfikacji integralności tokenu i autentyczności nadawcy.

Przykładowy token JWT:

```
1 eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjMONTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0Ij0iYWRtaW4iOnRydWUsImVudCI6MTUxNjIzOTYyMn0.KMUFsIDTnFmyG3nMiGM6H9FNFUR0f3wh7SmqJp-QV30
```

Nagłówek przedstawia zakodowane informacje o algorytmie podpisu oraz typie tokenu (w tym przypadku HS256):

```
1 {
2   "alg": "HS256",
3   "typ": "JWT"
4 }
```

Payload zawiera zakodowane informacje, które chcemy przekazać, jak np. identyfikator użytkownika, nazwa użytkownika, role itp.:

```
1 {
2   "ID": "76dc77d7-8dfc-4c79-8426-4d40568bd612",
3   "nickname": "miksiu120",
4   "role": "admin",
5   "token-type": "user",
6   "iat": 1516239022
7 }
```

Podpis jest generowany przez zakodowanie nagłówka i payloadu oraz podpisanie ich kluczem prywatnym przy użyciu algorytmu określonego w nagłówku.

W środowisku .NET mechanizm tokenów JWT jest wspierany przez bibliotekę `Microsoft.AspNetCore.Authentication`, która umożliwia łatwą integrację z mechanizmami uwierzytelniania i autoryzacji w aplikacjach ASP.NET Core. Dzięki temu jesteśmy w stanie generować w efektywny sposób tokeny, które wykorzystywane są do autoryzacji.

```

1 var claims = new List<Claim>
2 {
3     new(ClaimTypes.NameIdentifier, user.Id.ToString()),
4     new(ClaimTypes.Email, user.Email),
5     new(ClaimTypes.Role, user.Role),
6     new(ClaimTypes.Name, user.Nickname),
7     new("token_type", "user")
8 };
9
10 // Klucz podpisujący token (HMAC SHA-256)
11 var key = new SymmetricSecurityKey(
12     Encoding.UTF8.GetBytes(_authenticationSettings.JwtKey));
13
14 // Świadczanie podpisu
15 var credentials = new SigningCredentials(
16     key, SecurityAlgorithms.HmacSha256);
17
18 // Czas wygaśnięcia tokenu
19 var expires = DateTime.Now.AddMinutes(
20     _authenticationSettings.JwtExpireAccount);
21
22 // Utworzenie obiektu tokenu JWT
23 var token = new JwtSecurityToken(
24     issuer: _authenticationSettings.JwtIssuer,
25     audience: _authenticationSettings.JwtIssuer,
26     claims: claims,
27     expires: expires,
28     signingCredentials: credentials
29 );

```

Listing 8.10: Generowanie tokenu JWT w warstwie backendu

Identyfikowanie użytkowników i autoryzacja dostępu do zasobów

W aplikacji wykorzystano mechanizm autoryzacji oparty na atrybutach [Authorize] dostępnych w ASP.NET Core. Atrybut ten pozwala na zabezpieczenie kontrolerów lub konkretnych akcji, wymagając od użytkownika posiadania ważnego tokenu JWT oraz odpowiednich uprawnień (ról) do dostępu do zasobów. Na etapie przetwarzania żądania, middleware uwierzytelniające sprawdza obecność i ważność tokenu JWT w nagłówku autoryzacji. Jeśli token jest poprawny, użytkownik zostaje zidentyfikowany, a jego rola jest sprawdzana w kontekście żądanej akcji.

Hashowanie haseł użytkowników i uwierzytelnianie

Do bezpiecznego generowania i przechowywania haseł użytkowników wykorzystano mechanizm `PasswordHasher<TUser>` udostępniony przez .NET Core Identity. Hasher ten jest odporny na ataki brute-force dzięki zastosowaniu kosztownego obliczeniowo procesu hashowania oraz losowego saltingu. W procesie rejestracji hasło użytkownika jest hashowane przed zapisem do bazy danych, natomiast podczas logowania następuje weryfikacja hasła poprzez porównanie jego hasha z wartością przechowywaną w bazie.

```

1 // Hashowanie hasła podczas rejestracji
2 var passwordHash = passwordHasher.HashPassword(

```

```

3     newUser,
4     registerAccountCommand.Password
5 );
6
7 // Weryfikacja hasła podczas logowania
8 var result = passwordHasher.VerifyHashedPassword(
9     user,
10    user.PasswordHash,
11    loginCommand.Password
12 );

```

Listing 8.11: Hashowanie i weryfikacja hasła użytkownika

8.2.3. Walidacja danych

W celu zapewnienia poprawności danych wejściowych w aplikacji zastosowano bibliotekę *FluentValidation*. Umożliwia ona definiowanie reguł walidacyjnych w sposób deklaratywny, czytelny i łatwy w utrzymaniu. Przykładowo, podczas rejestracji użytkownika można zdefiniować reguły dotyczące minimalnej i maksymalnej długości hasła, prawidłowego formatu adresu e-mail oraz unikalności nazwy użytkownika w systemie. Dzięki temu możliwe jest centralne zarządzanie walidacją oraz zapewnienie spójności danych w całej aplikacji.

Biblioteka *FluentValidation* jest zintegrowana z mechanizmem wiązania modeli (model binding) w ASP.NET Core, co umożliwia automatyczne wywoływanie walidacji podczas przetwarzania żądań HTTP. W przypadku wykrycia błędów walidacji framework automatycznie zwraca odpowiedź HTTP 400 (Bad Request) wraz ze szczegółowym opisem napotkanych problemów, co znacząco ułatwia debugowanie i poprawia komunikację z klientem API.

Niektóre reguły walidacyjne wymagają dostępu do warstwy persystencji, na przykład w celu sprawdzenia, czy w bazie danych nie istnieje już użytkownik o podanym adresie e-mail lub nazwie. W takich przypadkach do konstruktora walidatora wstrzykiwane są odpowiednie repozytoria poprzez mechanizm wstrzykiwania zależności (Dependency Injection).

```

1 public class RegisterUserDtoValidator : AbstractValidator<RegisterUserDto>
2 {
3     private readonly IAccountRepository _accountRepository;
4
5     public RegisterUserDtoValidator(IAccountRepository accountRepository)
6     {
7         _accountRepository = accountRepository;
8         RuleFor(u => u.Email)
9             .NotEmpty()
10            .EmailAddress();
11
12        RuleFor(u => u.Nickname)
13            .NotEmpty()
14            .MinimumLength(3)
15            .MaximumLength(25);
16
17        RuleFor(u => u.Password)
18            .NotEmpty()

```

```

19         .MinimumLength(8)
20         .WithMessage("Minimum_length_of_password_is_8");
21
22     RuleFor(x => x.ConfirmPassword)
23         .Equal(e => e.Password)
24         .WithMessage("Passwords_are_not_the_same");
25 }
26 }

```

Listing 8.12: Przykład walidacji danych użytkownika podczas rejestracji

Dzięki takiemu podejściu walidacja danych jest centralnie zarządzana i łatwo rozszerzalna, co przekłada się na wyższą jakość oraz bezpieczeństwo aplikacji. Każda reguła walidacyjna znajduje się w osobnej klasie, co pozwala zachować zasadę pojedynczej odpowiedzialności, zwiększa modularność kodu oraz umożliwia łatwe testowanie poszczególnych walidatorów w izolacji od pozostałych komponentów systemu.

8.2.4. Obsługa błędów w aplikacji

W aplikacji wykorzystano globalne middleware w ASP.NET Core, odpowiedzialne za przechwytywanie wyjątków w dowolnym miejscu w kodzie. Middleware konwertuje je na spójne odpowiedzi HTTP z odpowiednim kodem statusu oraz komunikatem błędu — dzięki temu użytkownik otrzymuje czytelne informacje, bez ujawniania stosu wywołań ani szczegółów serwera. Cały mechanizm dostępny jest w jednym miejscu, co ułatwia zarządzanie błędami i poprawia doświadczenie użytkownika.

Dodatkowo, dla każdej encji w warstwie domeny zdefiniowano osobny plik z wyjątkami specyficznymi dla danej encji. Poniżej pokazano przykład wyjątku domenowego dla encji użytkownika:

Przykład definicji wyjątku domenowego dla encji użytkownika przedstawiono poniżej:

```

1
2 // Definicja przykładowego wyjątku
3 public class UserNotFoundException(Guid userGuid) :
4     BaseException($"User_{GUID}_{userGuid}_was_not_found")
5 {
6     public override int StatusCode => 404;
7 }
8 // Rzucanie wyjątku w przypadku braku użytkownika
9 throw new UserNotFoundException(userGuid);

```

Listing 8.13: Przykład definicji wyjątku domenowego dla encji użytkownika

Dzięki takiemu podejściu możliwe jest precyzyjne zarządzanie typerami błędów typowymi dla konkretnej encji, co ułatwia debugowanie i podnosi jakość obsługi wyjątków.

8.3. Baza danych i zarządzanie danymi

8.3.1. Repozytoria — komunikacja z bazą danych

W celu oddzielenia logiki biznesowej od warstwy dostępu do danych zastosowano wzorzec repozytorium. Jeśli obiekt chce uzyskać dostęp do danych, zamiast komunikować się bezpośrednio z bazą danych, korzysta z interfejsu repozytorium. Repozytorium udostępnia metody do wykonywania operacji CRUD (Create, Read, Update, Delete) na encjach, ukrywając szczegóły implementacji dostępu do

danych. Dzięki temu logika biznesowa pozostaje niezależna od konkretnej technologii bazy danych, co ułatwia testowanie i utrzymanie kodu.

W aplikacji wszystkie repozytoria dziedziczą po wspólnym interfejsie bazowym `IRepository<T>`, który definiuje podstawowe operacje CRUD. Każde repozytorium specyficzne dla danej encji rozszerza ten interfejs, dodając metody charakterystyczne dla konkretnej domeny. Przykładowo, repozytorium użytkowników może zawierać dodatkowe metody do wyszukiwania użytkowników według adresu e-mail lub nazwy użytkownika.

8.3.2. Warstwa persystencji — *Entity Framework Core*

Wykorzystując Entity Framework jako ORM (Object-Relational Mapping), w pierwszej kolejności zaimplementowano modele encji odpowiadające tabelom w bazie danych. Następnie została zdefiniowana klasa dziedzicząca po `DbContext` — klasie, w której definiujemy związki i relacje wykorzystywane przy mapowaniu, ustalając przy tym zasady działania kaskadowego usuwania powiązanych encji.

Chcąc zaktualizować bazę danych, stosujemy migracje, które pozwalają na dodawanie, usuwanie i modyfikowanie istniejących już definicji obiektów. Ponadto dzięki nim możemy wersjonować stan bazy, aby w razie błędu cofnąć się do poprzedniej definicji. Za pomocą polecenia `add-migration <nazwa_migracji>` tworzymy nową migrację, a następnie poleceniem `update-database` stosujemy zmiany w bazie danych. W folderze `Migrations` przechowywane są pliki migracji, które zawierają instrukcje dotyczące zmian w strukturze bazy danych. Są one generowane automatycznie i mają strukturę nazwy `<timestamp>_<nazwa_migracji>.cs`.

8.3.3. Adminer

W celu podglądu i zarządzania bazą danych PostgreSQL w środowisku deweloperskim wykorzystano narzędzie Adminer. Adminer to lekka aplikacja webowa napisana w PHP, która umożliwia łatwe przeglądanie, edytowanie i zarządzanie bazami danych poprzez interfejs przeglądarki internetowej.

8.3.4. Problemy napotkane przy implementacji backendu

-Eager loading vs lazy loading w EF Core
-Konflikty migracji bazy danych
-Obsługa błędów w aplikacji ASP .NET Core
-Debugowanie aplikacji w kontenerze Docker

8.4. Devops i wdrożenie aplikacji

W tej sekcji opisano proces przygotowania, uruchamiania oraz wdrażania aplikacji w środowiskach deweloperskim i produkcyjnym z wykorzystaniem konteneryzacji Docker, plików konfiguracyjnych oraz reverse proxy Nginx. Przedstawiono również decyzje projektowe dotyczące zarządzania certyfikatami oraz publikacji obrazów w zewnętrznym rejestrze (Docker Hub).

8.4.1. Środowisko deweloperskie

Środowisko deweloperskie zostało zaprojektowane tak, aby umożliwić szybkie iteracje oraz łatwe debugowanie. Kluczowe założenia:

- Oddzielne kontenery dla frontendu (React/TypeScript) i backendu (ASP.NET Core) uruchamiane w trybie `watch/hot reload`.
- Kontener bazy danych PostgreSQL oraz (opcjonalnie) narzędzie Adminer do inspekcji danych.
- Użycie pliku `docker-compose.dev.yaml` do orkiestracji wielokontenerowego środowiska.

Konfiguracja usług w pliku `compose` dla środowiska deweloperskiego może zawierać sekcje z wolumenami mapowanymi na katalogi źródłowe, aby kod był automatycznie odświeżany. Backend korzysta z `dotnet watch run`, frontend z `npm start`. Daje to krótki cykl: edycja → zapis → natychmiastowy efekt. Eliminuje to długie czasy rekompilacji całej aplikacji.

Debugowanie w kontenerze Dla backendu możliwe jest podłączenie zdalnego debuggera (np. VS Code / Rider) przez port udostępniony w kontenerze. W środowisku deweloperskim certyfikaty HTTPS nie są wymagane, ponieważ komunikacja odbywa się po `http://localhost`.

8.4.2. Środowisko produkcyjne

W środowisku produkcyjnym nacisk położono na izolację usług, bezpieczeństwo i skalowalność. Kluczowe elementy:

- Użycie pliku `docker-compose.prod.yaml` z obrazami już zbudowanymi (bez mountowania kodu źródłowego).
- Wprowadzenie reverse proxy Nginx jako pojedynczego punktu wejścia (terminacja TLS, routing do usług wewnętrznych).
- Minimalizacja rozmiaru obrazów przez wykorzystanie wieloetapowych buildów (multi-stage) w Dockerfile backendu i frontendu.
- Brak narzędzi administracyjnych (np. Adminer) w docelowym środowisku - uruchamiane jedynie tymczasowo przy potrzebie migracji lub analizy.

8.4.3. Pliki `docker-compose` i ich role

W projekcie zastosowano kilka wariantów plików `docker-compose`. Mają one strukturę kaskadową: `docker-compose.infra.yaml` Definicje infrastrukturalne (baza danych, narzędzia pomocnicze). Pozwala odseparować warstwę aplikacyjną od zasobów zewnętrznych.

`docker-compose.dev.yaml` Konfiguracja zawierająca część frontendową oraz infrastrukturalną, umożliwia uruchomienie backendu poza kontenerem.

`docker-compose.yaml` Plik zawierający wszystkie powyższe rzeczy, gotowy do uruchomienia przez członków zespołu bez zbędnej konfiguracji.

`docker-compose.prod.yaml` Uporządkowane obrazy z tagami wersji, brak wolumenów źródłowych, zmienne środowiskowe wczytywane z plików `.env`, integracja z Nginx.

co pozwala na nadpisywanie sekcji (*override*) dla danego środowiska.

8.4.4. Decyzja o użyciu reverse proxy Nginx

Pierwotnie rozważano generowanie i instalację certyfikatów HTTPS w każdej części aplikacji osobno (frontend i backend). Takie rozwiązanie zwiększało złożoność utrzymania (odnawianie, dystrybucja, synchronizacja dat ważności). Zamiast tego wprowadzono warstwę Nginx jako reverse proxy z terminacją TLS:

- Jeden certyfikat (np. Let's Encrypt) dla domeny głównej.
- Uproszczona konfiguracja `appsettings.json` - backend może działać na HTTP wewnątrz sieci Docker.

- Brak potrzeby modyfikowania konfiguracji frontendu przy odnowieniu certyfikatu - zmiana dotyczy wyłącznie proxy.
- Możliwość dodania reguł bezpieczeństwa (rate limiting, nagłówki CSP, HSTS) w jednym miejscu.

Korzyści architektoniczne Centralizacja TLS redukuje liczbę punktów potencjalnych błędów i przyspiesza automatyzację (skrypt odnawiający certyfikat uruchamiany cyklicznie w kontenerze certbot).

8.4.5. Certyfikaty i ich zarządzanie

W katalogu `https/` znajdował się lokalny certyfikat deweloperski `dev-cert.pfx` używany w fazie wczesnego rozwoju do testów lokalnych. Po wdrożeniu Nginx rola tego pliku została ograniczona wyłącznie do scenariuszy deweloperskich. W produkcji certyfikaty są pobierane automatycznie (np. przez certbot) i przechowywane w zamontowanym wolumenie na hosta, dzięki czemu ich odnowienie nie wymaga przebudowy obrazów.

8.4.6. Budowa i publikacja obrazów Docker

Proces publikacji obrazu na Docker Hub obejmuje kroki:

1. Budowa obrazu (np. `docker build -t użytkownik/nazwa:tag .`).
2. Logowanie do rejestru (`docker login`).
3. Wysłanie obrazu (`docker push użytkownik/nazwa:tag`).
4. Opcjonalne użycie wersjonowania semantycznego (1.2.0, latest).

Multi-stage build pozwala oddzielić etap kompilacji (cięższy obraz z SDK) od etapu wykonawczego (lżejszy runtime). Zmniejsza to czas pobierania i powierzchnię ataku.

Tagowanie i automatyzacja Przy każdej zmianie głównej funkcjonalności można generować nowy tag. W przyszłości możliwe jest dodanie pipeline CI (GitHub Actions) automatyzującej budowę i publikację na podstawie tagów w repozytorium Git.

8.4.7. Konfiguracja aplikacji

Konfiguracja jest rozproszona pomiędzy kilka plików i mechanizmów:

- Pliki `.env` Zawierają wrażliwe lub zmienne środowiskowe (np. connection string do PostgreSQL, sekrety JWT). W plikach compose referencje przez `${NAZWA_ZMIENNEJ}`.
 - `appsettings.json`, `appsettings.Development.json`, `appsettings.Production.json` Warstwowe źródło konfiguracji backendu: logowanie, połączenie do bazy, parametry JWT (issuer, audience, czas życia), ustawienia CORS.
 - `launchSettings.json` Używany lokalnie (poza Docker) do definiowania profili uruchomieniowych i portów (HTTPS/HTTP). W produkcji zastąpione przez konfigurację kontenera + reverse proxy.
 - `Frontend Constants.ts` Zawiera stałe takie jak URL API, nazwy eventów, parametry mapy. W produkcji wartości mogą być nadpisane przez zmienne środowiska przy budowie (np. `VITE_API_URL`).
 - Zmienne środowiskowe kontenerów Nadawane w plikach compose lub w CI/CD (np. `ASPNETCORE_ENVIRONMENT=Pr`)
- Mechanizm konfiguracji w ASP.NET Core scalający źródła (pliki, zmienne środowiskowe, sekrety użytkownika) umożliwia elastyczne przełączanie środowisk bez modyfikacji kodu.

Bezpieczeństwo konfiguracji Sekrety (klucz do podpisu JWT, hasło do bazy) nie powinny być wersjonowane — w środowisku deweloperskim mogą być przechowywane w pliku `.env.local` dodanym do `.gitignore`. W produkcji zalecane jest ich wstrzykiwanie poprzez menedżera sekretów (np. HashiCorp Vault / Docker Swarm secrets / Kubernetes secrets).

8.4.8. *Przepływ uruchomienia aplikacji*

Cały proces startu w produkcji można streścić następująco:

1. Nginx startuje i ładuje certyfikat TLS.
2. Backend uruchamia się w trybie HTTP wewnątrz sieci Docker, migracje bazy wykonywane automatycznie na starcie (opcjonalnie sterowane flagą).
3. Frontend serwowany jako statyczne pliki (np. z `nginx`) lub z dedykowanego kontenera serwera plików.
4. Nginx przekazuje żądania `/api` do backendu, a pozostałe ścieżki obsługuje przez serwowanie SPA.

8.4.9. *Podsumowanie sekcji DevOps*

Zastosowana architektura DevOps upraszcza cykl życia aplikacji: deweloperzy otrzymują szybkie iteracje dzięki mountowaniu kodu, produkcja korzysta z odchudzonych i bezpiecznych obrazów oraz centralnego zarządzania TLS. Decyzja o użyciu reverse proxy Nginx z jednym certyfikatem poprawiła bezpieczeństwo i zmniejszyła nakład operacyjny, tworząc solidną bazę pod dalsze skalowanie i obserwowalność.

8.5. *Integracja komponentów systemu*

9. TESTOWANIE

10. PODSUMOWANIE

11. MOŻLIWOŚĆ DALSZEGO ROZWOJU

WYKAZ RYSUNKÓW

3.1	Gra Travian wydana w 2004 roku.	10
3.2	Gra Club Penguin wydana w 2005 roku.	11
3.3	Gra Agar.io wydana w 2015 roku.	11
3.4	Gra Slither.io wydana w 2016 roku.	11
3.5	Gra Geoguessr wydana w 2020 roku.	12
3.6	Gra PokemonGO wydana w 2016 roku.	13
3.7	Gra Geotastic wydana w 2020 roku.	13
3.8	Gra Back of Your Hand wydana w 2020 roku.	14
7.1	Uproszczony schemat architektury systemu UniGuesser – ogólny podział na warstwy .	29
7.2	Schemat komunikacji przy zastosowaniu wzorca CQRS	30
7.3	Diagram ERD dla bazy danych aplikacji UniGuesser	31
7.4	Diagram przypadków użycia aplikacji UniGuesser	34
7.5	Diagram sekwencji przedstawiający przebieg rozgrywki w aplikacji UniGuesser	35
7.6	Przykładowy ekran rejestracji użytkownika w aplikacji UniGuesser, ilustrujący zastosowaną kolorystykę	36

WYKAZ TABEL

7.1	Tabela Users	31
7.2	Tabela GameSessions	32
7.3	Tabela Rounds	32
7.4	Tabela Places	33